



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Corso di Laurea Triennale in Ingegneria Informatica

Tesi di Laurea

**Multi-ScaleGAN per Audio Super-Resolution:
ricostruzione di segnali musicali con RNN e
Discriminatore Multi-Scala**

Relatore:

Prof. Luigi Palopoli

Dott.ssa Simona Nisticò

Candidato:

Christian Bruni

Mat. 240008

ANNO ACCADEMICO 24/25

“In a way failure is a beautiful thing, because when the dust settles there’s nowhere to go but up, and it’s a freedom. You can’t lose more, but you can gain.”

- David Lynch

Introduzione

L'Audio Super Resolution è una innovativa tecnica nell'ambito dell'elaborazione di segnali audio digitali che mira a migliorarne sensibilmente la qualità percepita mediante l'utilizzo di estensioni della banda o di espansione della fase. Questa tecnica è utilizzata maggiormente nei casi in cui la traccia contenga le frequenze a banda bassa come nel caso di molte registrazioni ottenute tramite strumenti datati o di bassa qualità. L'obiettivo del lavoro è quello di creare una Generative Adversarial Network che, facendo uso di un Generatore basato su Reti Ricorrenti e di un Discriminatore multi-scala, possa ricostruire con una elevata fedeltà le frequenze mancanti dagli audio passati in input.

Nel primo capitolo si introduce il concetto di reti neurali, la loro storia, il loro funzionamento, le varie tipologie di reti note con un approfondimento sulle Generative Adversarial Networks.

Nel secondo capitolo vengono trattati i concetti fondamentali dell'elaborazione dei segnali digitali, le tecniche e gli strumenti usati in questo ambito con un approfondimento sulla ricostruzione dei segnali.

Nel terzo capitolo si introduce il problema dell'Audio Super Resolution, i modelli proposti in letteratura e alcune delle metriche di valutazione usate in questo ambito.

Nel quarto capitolo viene trattata la parte di sperimentazione con la presentazione di una Generative Adversarial Network per Audio Super Resolution costituita da una Rete Ricorrente e da un Discriminatore Multi-Scala.

Indice

Indice	3
1 Introduzione alle Reti Neurali ed alle Generative Adversarial Networks	6
1.1 Il Neurone Artificiale	6
1.1.1 Le funzioni di attivazione	6
1.2 Le Reti Neurali Artificiali	8
1.3 Le Reti Convoluzionali	9
1.3.1 Dimensione dell'output	10
1.3.2 Riduzione della dimensionalità dell'output	10
1.3.3 Architetture tipiche	11
1.4 La fase di apprendimento	11
1.4.1 La funzione di costo	11
1.4.2 Scelta delle funzioni di costo	12
1.4.3 Discesa del gradiente	13
1.4.4 Backpropagation	15
1.5 Introduzione alle Generative Adversarial Networks	15
1.5.1 Addestramento delle GANs	16
1.5.2 Stabilità delle GANs	18
1.5.3 Wasserstein Generative Adversarial Networks	19
1.5.4 Two time-scale update rule	20
1.5.5 Valutazione delle prestazioni	21
1.5.6 Fréchet Inception Distance	21

2	Introduzione al Digital Signal Processing	23
2.1	Concetti Fondamentali	23
2.1.1	Digital Signal Processing	23
2.1.2	Campionamento e Quantizzazione	23
2.1.3	Codifica nel Dominio del Tempo e della Frequenza	24
2.2	Rappresentazioni del segnale audio	26
2.2.1	Pulse-Code Modulation Lineare	26
2.2.2	Companding	27
2.3	La Short-Time Fourier Transform	28
2.3.1	Rappresentazione matematica	29
2.3.2	Funzione di finestra	29
2.3.3	Finestra Rettangolare (o Finestra di Dirichlet)	30
2.3.4	Finestra Triangolare (o Finestra di Fejer-Bartlet)	30
2.3.5	Finestra $\cos^\alpha(X)$	31
2.3.6	Finestra di Hamming	32
2.3.7	Mel-spectrogram	33
2.4	Tecniche di Ricostruzione	35
2.4.1	Filter Bank Summation	35
2.4.2	Overlap Addition	36
2.4.3	Frequenze di campionamento	38
2.4.4	Campionamento nel tempo	38
2.4.5	Campionamento in frequenza	38
2.4.6	Dualità tra FBS e OLA	39
3	Audio Super Resolution	40
3.1	Definizione del problema	41
3.2	Modelli proposti	42
3.2.1	Architettura di Kuleshov et al.	42
3.2.2	Architettura WSRGlow	43
3.2.3	Architettura AERO	48
3.2.4	Architettura Vit-SR	50

3.3	Metriche di valutazione per Audio Super Resolution	52
3.3.1	Signal-to-Noise ratio	52
3.3.2	Log-spectral Distance	53
4	Sperimentazione	54
4.1	Reti Neurali Ricorrenti	54
4.2	Architetture con molteplici Discriminatori	55
4.2.1	Architettura GMAN	55
4.2.2	Discriminatori multi-scala per immagini	57
4.2.3	Discriminatori multi-scala per audio	58
4.3	Specifiche di sperimentazione	61
4.3.1	Il Free Music Archive dataset	61
4.3.2	Implementazione del Discriminatore multi-scala per Audio Super Resolution	61
4.3.3	SubDiscriminator	61
4.3.4	MultiScaleDiscriminator	62
4.3.5	Specifiche dell'addestramento	62
4.4	Risultati	64
5	Conclusioni	66
	Bibliografia	68

Capitolo 1

Introduzione alle Reti Neurali ed alle Generative Adversarial Networks

1.1 Il Neurone Artificiale

Introdotta per la prima volta nel 1943 da W.S. McCulloch e W. Pitts, il Neurone Artificiale ha rappresentato il primo tentativo, da parte dell'uomo, di formalizzare il comportamento del neurone biologico come una funzione booleana [1]. Il modello è costituito da una funzione non lineare, detta "di attivazione", che trasforma la somma ponderata tra variabili in input x e il vettore di pesi w , in un output y , secondo la formula:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Dove x_i è l' i -esimo elemento del vettore di input reali $(x_1, x_2, \dots, x_n)$, w_i è l' i -esimo elemento del vettore di *pesi* $(w_1, w_2, \dots, w_n)$ che sono associati alle variabili in ingresso ed i cui valori saranno successivamente aggiornati durante la fase di apprendimento. In ultimo, il termine b , noto anche come "bias" (o "soglia"), permette l'attivazione del neurone anche in presenza di input nulli. [2]

1.1.1 Le funzioni di attivazione

La scelta di usare funzioni non lineari diventa chiara quando si pensa all'esistenza di possibili relazioni non lineari tra i dati in input, il loro impiego diventa dunque

indispensabile se si vuole permettere alla rete di cogliere tali aspetti. La scelta della funzione di attivazione è dettata dal tipo di task che il neurone dovrà affrontare, tra le più note e utilizzate annoveriamo, ad esempio, la funzione sigmoide, così definita:

$$y = \frac{1}{1 + e^{-x}}$$

estremamente utile in quanto gode delle seguenti proprietà:

- $\lim_{x \rightarrow \infty} \sigma(x) = 1$;
- $\lim_{x \rightarrow -\infty} \sigma(x) = 0$;
- $\sigma(-x) = 1 - \sigma(x)$;

Ciò permette al neurone di svolgere il ruolo di classificatore binario, poiché gli output della funzione sigmoide sono compresi nell'intervallo (0,1).

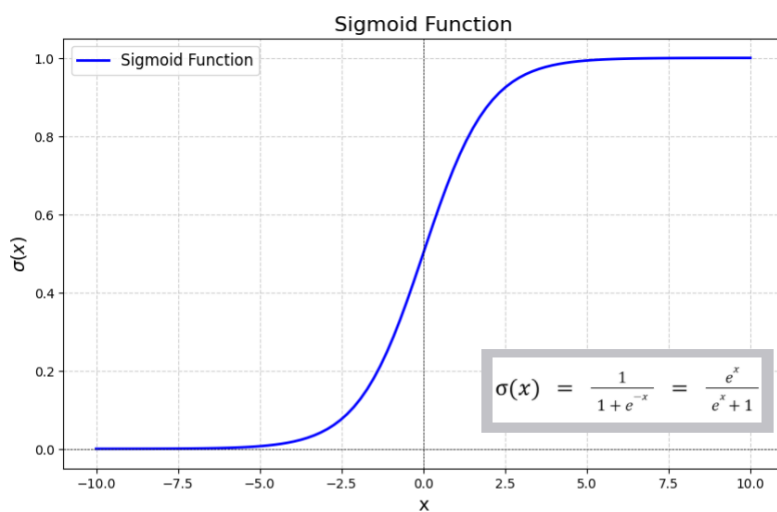


Figura 1.1: Grafico della funzione di attivazione sigmoide

Inoltre, Cybenko [3] ha dimostrato che le reti feedforward con un solo layer nascosto che fanno uso della funzione sigmoide possono approssimare arbitrariamente bene qualsiasi funzione continua. Altre funzioni di attivazione sono le *funzioni rettificatore* come ReLu, definita come:

$$ReLU(x) = x^+ = \max(0, x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0, \\ 0 & x \leq 0 \end{cases}$$

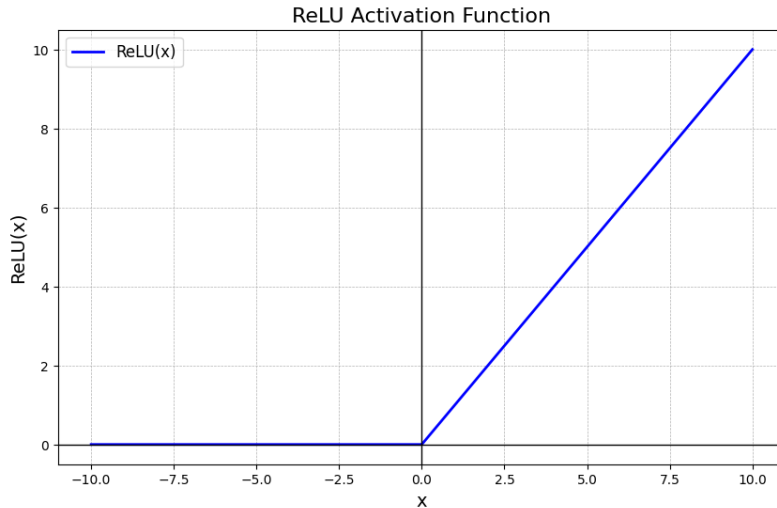


Figura 1.2: Grafico della funzione di attivazione ReLU

Introdotta da Householder nel 1941 [4] come astrazione dell'attività stazionaria dei fasci di fibre nervose, questa funzione modella l'intensità dell'attività neuronale che cresce linearmente con $x - \theta$, dove x rappresenta l'input e θ rappresenta invece un parametro soglia. Verrà usata nell'ambito delle reti neurali da K. Fukushima nel 1969 [5] per l'estrazione di caratteristiche visive in una pionieristica architettura basata sugli studi sulla corteccia visiva dei mammiferi. Il singolo neurone artificiale costituisce l'elemento fondamentale per costruire strutture sempre più complesse: le interazioni tra più neuroni permettono infatti la creazione delle Reti Neurali.

1.2 Le Reti Neurali Artificiali

Le reti neurali artificiali (o ANN) sono sistemi complessi basati sulle connessioni tra neuroni artificiali organizzati secondo una struttura a strati. Queste si differenziano sulla base di scelte progettuali come il numero di strati nascosti o sulle modalità in cui un singolo nodo può propagare il proprio output [2]. La forma di rete neurale più semplice è la rete *feedforward*, composta da tre strati fondamentali:

- Uno strato di ingresso che riceve i dati di input;

- Uno strato nascosto, come nel caso delle Shallow Neural Networks, o più strati nascosti, come nel caso delle Deep Neural Networks, dove avviene l'elaborazione intermedia dei dati tramite le funzioni di attivazione;
- Uno strato di uscita che restituisce le predizioni della rete;

Una rete è definita "completamente connessa" se ogni neurone presente nello strato i -esimo è connesso a tutti i neuroni dello strato $i+1$ -esimo [2]. Una tipologia di rete derivata dalle reti feedforward è la Rete Convoluzionale.

1.3 Le Reti Convoluzionali

Le Reti Convoluzionali (CNN) sono un modello di rete neurale profonda ispirate dal funzionamento della corteccia visiva, queste infatti sono progettate per elaborare dati con struttura a griglia come le immagini. Questo tipo di architettura fu presentata per la prima volta da K. Fukushima nel 1980, nella forma della rete "Neocognitron" [6]. A differenza delle reti neurali feedforward, le CNN possiedono uno *strato di convoluzione* che permette l'estrazione di pattern dai dati: viene fatto scorrere sull'input un filtro bidimensionale, chiamato anche *kernel*, e per ogni posizione viene eseguito il prodotto elemento per elemento tra i valori del kernel e gli elementi della regione su cui si trova il filtro. I risultati delle singole operazioni vengono poi sommati per restituire un solo valore in output [6]. La ripetizione del procedimento nei vari strati permette di estrarre una *feature map* a partire dagli ingressi. La convoluzione bidimensionale viene espressa come:

$$Y[i, j] = \sum_{m=0}^{h-1} \sum_{n=0}^{w-1} X[i + m, j + n] K[m, n]$$

dove X è il tensore in input, K è il kernel di dimensione $h \times w$ applicato al dato e Y è la feature map risultante. Una caratteristica delle CNN è quella della condivisione dei pesi, di fatto, lo stesso kernel è applicato in tutte le posizioni dell'input e ciò permette di avere un minore numero di pesi da gestire e consente di rilevare pattern simili in qualsiasi punto dell'immagine [2].

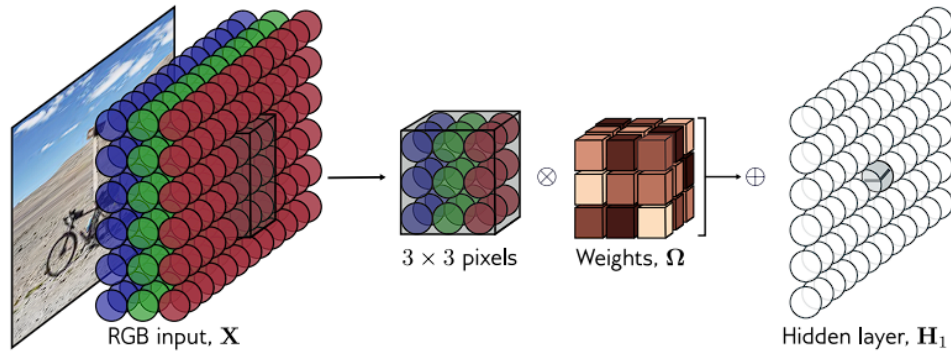


Figura 1.3: Un esempio di rete convoluzionale bidimensionale

1.3.1 Dimensione dell'output

L'output è costituito da una mappa di feature la cui dimensione dipende sia dai dati in input, sia da una coppia di iperparametri: *stride* e *padding*. Il primo definisce di quanti pixel si sposta il filtro in ogni passo della convoluzione, ed è immediato comprendere che a valori elevati di stride corrispondono fenomeni di *sotto-campionamento* della mappa di output, mentre per il valore 1 corrisponderà applicare il filtro in ogni posizione dell'immagine. Il padding consiste invece nell'aggiungere, ai bordi dell'immagine, pixel (tipicamente con valore 0) per controllare la dimensione della mappa di output [2]. La dimensione della mappa è dunque così calcolata:

$$O = \frac{I - F + 2P}{S} + 1.$$

dove I rappresenta la dimensione spaziale dell'input, F la dimensione del kernel, P ed S rispettivamente i valori di padding e di stride.

1.3.2 Riduzione della dimensionalità dell'output

Nelle più comuni architetture per Reti Convoluzionali lo strato di convoluzione è seguito da uno strato di *pooling*, anch'esso introdotto per la prima volta nel Neocognitron [6], all'interno del quale è possibile ridurre ulteriormente la dimensione dell'output. L'operazione di pooling consiste nel sotto-campionamento della feature map secondo alcune logiche, le più comuni sono *max pooling* ed *average pooling*. In entrambi i casi la

mappa è divisa in *finestre* di dimensione fissata, e per ogni finestra viene estratto un solo valore: nel caso del max pooling sarà estratto il valore massimo, mentre nell'average il valore medio. Scegliendo accuratamente le dimensioni delle finestre è possibile ridurre di molto la dimensione delle mappe in uscita [2].

1.3.3 Architetture tipiche

Come affermato in precedenza, le CNN sono caratterizzate dalla convoluzione e dal pooling, non sono però le uniche operazioni utilizzate all'interno di queste architetture. A seguito di strati di convoluzione e di pooling si possono trovare operazioni di *appiattimento* (o "flattening") che consentono ad un input di dimensione (H, W, C) di essere trasformato in un vettore unidimensionale con un numero di elementi pari a $H \times W \times C$. Una volta ottenuto il vettore è possibile darlo in input a strati *completamente connessi* per produrre l'output desiderato.

1.4 La fase di apprendimento

Come anticipato in precedenza, la rete ha necessità di evolvere per migliorare le sue prestazioni e questa operazione consiste nell'aggiornamento dei valori dei pesi durante l'apprendimento. In questa fase, gli input sono propagati in avanti di strato in strato attraverso le connessioni esistenti tra i singoli neuroni, ed una volta raggiunto lo strato di uscita viene computata una funzione di costo (o "loss") L che misura l'errore tra la predizione della rete $\hat{y}$ e il valore obiettivo y .

1.4.1 La funzione di costo

Considerato un dataset di addestramento $\{(x_i, y_i)\}_{i=1}^I$ costituito da coppie di input e output, $L[\phi]$ è una funzione "di costo" il cui valore restituito descrive l'inconsistenza tra la predizione del modello $f[x_i, \phi]$ e il suo output corrispondente y_i , fornendo dunque una misura dell'errore commesso durante l'esecuzione del task [2]. L'obiettivo diventa quello di trovare valori di ϕ che minimizzino tale funzione così da aumentare la verosimiglianza tra la predizione e l'output [2]. In termini probabilistici, $f[x, \phi]$ può

essere visto come un modello che stima la distribuzione di probabilità condizionata $Pr(y|x)$ ed in questa nuova ottica l'obiettivo diventa quello di avere una probabilità elevata per y_i all'interno della distribuzione $Pr(y_i|x_i)$, calcolata a partire dagli input x_i [2]. La scelta della distribuzione ricade tra quelle parametriche della forma $Pr(y|\theta)$ dove θ rappresenta uno o più parametri, permettendo alla rete di stimarli tramite l'equazione $\theta_i = f[x_i, \phi]$. Come affermato in precedenza, ϕ deve poter massimizzare la verosimiglianza che in questo caso corrisponde ad un valore elevato della probabilità condizionata $Pr(y_i|x_i)$ per ogni elemento del training set. Da questo ragionamento si ricava la seguente scrittura [2]:

$$\hat{\phi} = \arg \max_{\phi} \left[\prod_{i=1}^I Pr(y_i | f[x_i, \phi]) \right]$$

Questo implica sia che i dati siano identicamente distribuiti, sia che le singole distribuzioni $Pr(y_i|x_i)$ siano indipendenti, ma porta anche ad una considerazione: il risultato del prodotto di probabilità potrebbe essere molto piccolo e di conseguenza difficile da rappresentare con precisione [2]. E' possibile arginare questo inconveniente applicando alla funzione il logaritmo per poterne sfruttare le proprietà, ottenendo una scrittura equivalente alla precedente:

$$\hat{\phi} = \arg \max_{\phi} \left[\sum_{i=1}^I \log[Pr(y_i | f[x_i, \phi])] \right]$$

Questa forma è nota anche come "log-verosimiglianza" [2] ed è inoltre possibile convertirla in termini di problema di minimizzazione, ottenendo

$$\begin{aligned} \hat{\phi} &= \arg \min_{\phi} \left[- \sum_{i=1}^I \log[Pr(y_i | f[x_i, \phi])] \right] \\ &= \arg \min_{\phi} [L[\phi]] \end{aligned}$$

Questa trasformazione ci riporta al discorso trattato in precedenza riguardante la ricerca del valore ottimale del parametro ϕ per la minimizzazione della funzione di costo [2].

1.4.2 Scelta delle funzioni di costo

Una volta stabilita la natura delle funzioni di costo è naturale porsi la domanda su quale sia la migliore da utilizzare. La scelta dipende dal tipo di task che il nostro modello

dovrà affrontare: per problemi di classificazione binaria, cioè tutti quei problemi in cui è necessario stabilire l'appartenenza di un oggetto ad una tra due classi (spesso indicate con 0 e 1), è impiegata la *Binary Cross-Entropy Loss* (o BCE), la cui formula è:

$$L = -\frac{1}{N} \sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

La funzione misura l'errore nella predizione dell'appartenenza dell'elemento $\hat{y}$ alla classe positiva, penalizzando l'errore sulla base della confidenza della rete. Se questa predicesse con probabilità elevata l'appartenenza di $\hat{y}$ ad una classe ed il risultato si rivelasse poi incorretto, il valore di loss sarebbe elevato e i pesi potrebbero subire grossi cambiamenti. Ne consegue che nel caso di una predizione incorretta ma assegnata con probabilità più bassa, questa si ripercuoterebbe con meno intensità sui valori dei pesi [2]. Per task di regressione invece, dove l'output non è una probabilità ma è costituito dalla predizione di uno o più valori reali a partire dai dati in input, è frequente l'utilizzo del *Mean Squared Error* (o MSE), la cui formula è:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

La funzione misura l'errore medio tra le predizioni fornite in output dal modello e i valori obiettivo. Le singole differenze sono elevate al quadrato e questo permette di evitare tutti i casi in cui i valori positivi e negativi si possano compensare tra di loro, ma espone anche ad una maggiore penalizzazione nei casi in cui la distanza tra i due oggetti fosse molto elevata. Questo rende la funzione estremamente sensibile alla presenza di outliers che potrebbe portare ad un incorretto aggiornamento dei pesi [2].

1.4.3 Discesa del gradiente

L'aggiornamento dei pesi deve avvenire tenendo conto dei valori della funzione di costo perché questi danno misura dell'errore di predizione. La finalità della procedura è quella di minimizzare la funzione di costo, in maniera iterativa, calcolandone la derivata rispetto ai pesi ($\frac{\partial L}{\partial w}$). Questa operazione restituisce il valore del gradiente, cioè il vettore che punta sempre secondo la direzione di massima crescita della funzione, ed avviene successivamente l'aggiornamento dei pesi nella direzione *opposta* [2].

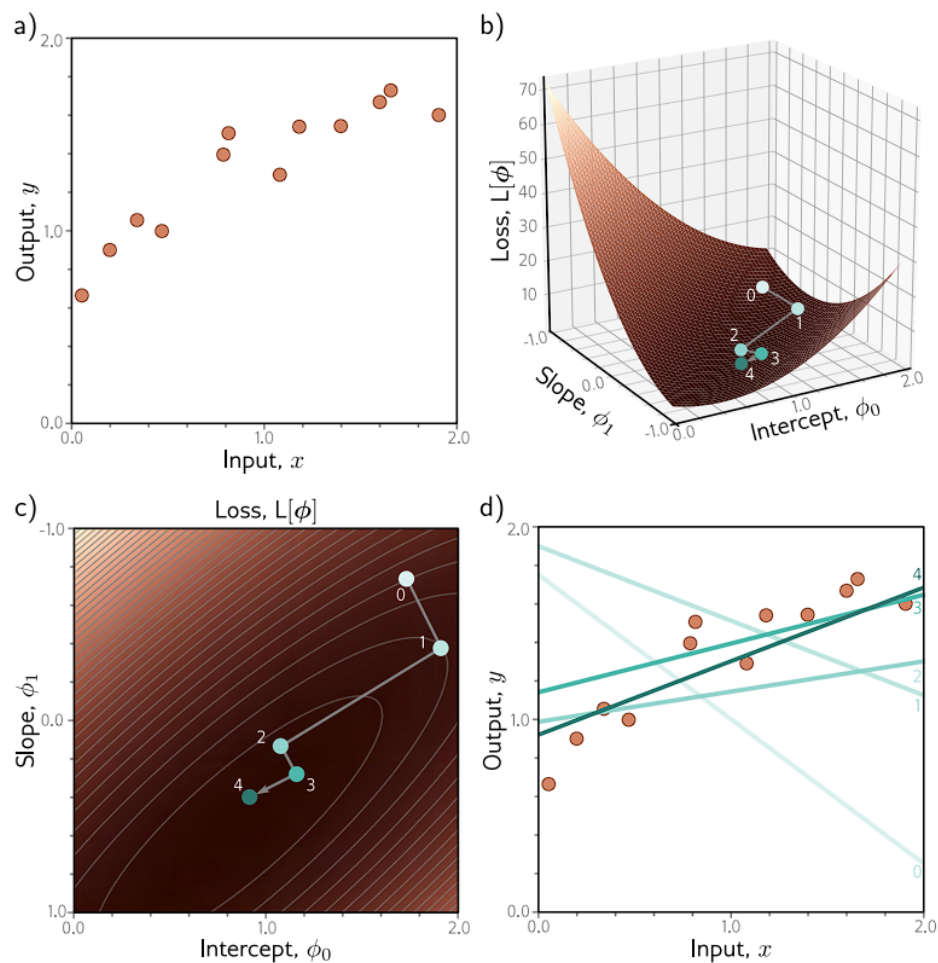


Figura 1.4: Esempio di discesa del gradiente per un modello di regressione lineare.

a) Training set di $I = 12$ coppie input/output (x_i, y_i) .

b) Iterazioni della discesa del gradiente sulla funzione di loss. Si parte dal punto 0 e ci si sposta verso un nuovo punto 1, l'operazione è ripetuta fino alla convergenza nel punto 4.

c) La discesa può essere visualizzata intuitivamente su una heatmap dove la luminosità indica la loss, si osserva come solo dopo 4 iterazioni si sia già raggiunto il minimo.

d) Il modello con i parametri al punto 0 (linea più chiara) descrive i dati in modo molto approssimativo, ma ogni iterazione successiva migliora l'adattamento. Il modello con i parametri al punto 4 (linea più scura) è già una descrizione ragionevole dei dati di addestramento. [2]

1.4.4 Backpropagation

L'operazione di aggiornamento dei pesi avviene nel contesto dell'algoritmo di backpropagation (o retro-propagazione), introdotto da Rumelhart et al. nel 1986 [7]. La procedura inizia con il passaggio dei dati in input all'interno della rete ed il conseguente calcolo della funzione di costo. Il passo successivo consiste nella propagazione (dal layer di uscita verso quello di ingresso) del valore di L , di cui sarà calcolata la derivata parziale per ogni peso all'interno della rete. Attraverso la regola della catena è possibile effettuare la propagazione degli errori layer per layer. L'approccio generalizza l'aggiornamento dei pesi dei singoli neuroni rendendolo scalabile ed applicabile nel contesto delle reti neurali profonde [7]. L'aggiornamento del generico peso w_{ij} dipenderà dalle derivate di tutti i pesi che lo "seguono": per ogni neurone all'interno della rete si definisce un "errore locale" δ_j , tale per cui vale la seguente equazione

$$\frac{\partial L}{\partial w_{ij}} = \delta_j x_i$$

La formula per l'aggiornamento dei pesi sarà dunque:

$$w_{ij} \leftarrow w_{ij} - \eta \delta_j x_i,$$

L'iperparametro η è noto come learning rate (o "tasso di apprendimento") e dal suo valore dipende quanto velocemente la rete convergerà: se questo avesse un valore troppo piccolo la rete convergerebbe lentamente; per valori troppo alti invece, la rete potrebbe non raggiungere mai il minimo. Risulta dunque essenziale la scelta oculata di un valore per η che eviti i casi estremi menzionati [2].

1.5 Introduzione alle Generative Adversarial Networks

Introdotte da Goodfellow et al. nel 2014 [8], le Generative Adversarial Networks (o GANs) rappresentano una novità nel panorama dei modelli generativi: l'intuizione è quella di coinvolgere due reti neurali distinte per l'addestramento, i due modelli sono chiamati G (o "Generatore") e D (o "Discriminatore"). Il Generatore dovrà apprendere come generare dati sintetici a partire da una distribuzione nota, mentre il Discriminatore dovrà distinguere i campioni reali dai campioni generati. La dinamica di questo tipo di

rete è impostata come un gioco di minimax tra due giocatori: Goodfellow et al. [8] descrivono questa situazione con l'analogia del "falsario" (G) che tenta di ingannare la "polizia" (D). Il generatore tenta di minimizzare $\log(1 - D(G(z)))$ mentre il discriminatore cercherà di massimizzare la probabilità di assegnare alla classe corretta sia le istanze reali che quelle sintetiche, questa dinamica è formalizzata come segue:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Il termine p_{data} rappresenta la distribuzione dei dati reali, p_z invece, la distribuzione del rumore in input al generatore. La teoria dei giochi fornisce gli strumenti per interpretare questo tipo di reti: i due giocatori coinvolti dovranno raggiungere l'equilibrio di Nash, cioè una configurazione di strategia per la quale nessuna delle due parti coinvolte può migliorare il proprio vantaggio cambiando unilateralmente strategia. Nel contesto delle GANs questo equilibrio è rappresentato da un punto di sella il cui raggiungimento non è sempre garantito, per questo motivo, l'addestramento può stabilizzarsi attorno ad un equilibrio locale, senza ovviamente convergere a $p_g = p_{data}$, ovvero quando la distribuzione dei dati generati coincide con quella dei dati reali [8].

1.5.1 Addestramento delle GANs

Le difficoltà che possono sorgere durante l'addestramento di una GAN sono diverse, in prima istanza vi è la necessità di alternare l'addestramento del Generatore e del Discriminatore. G dovrà inoltre aggiornare i propri parametri, non solo seguendo la propria funzione di costo, ma a questo valore dovrà essere aggiunto anche il valore della loss di D . Possono sorgere problemi di varia natura durante l'apprendimento: D potrebbe migliorare molto più rapidamente rispetto a G , di fatto bloccandone l'addestramento oppure il Generatore potrebbe incombere nel "collasso di modalità", condizione per la quale G si specializza nella riproduzione di una o poche varietà dei campioni che riescono ad ingannare D , perdendo la varietà offerta dalla distribuzione p_{data} . Risulta dunque fondamentale prestare estrema attenzione alla fase di apprendimento, pertanto la tecnica proposta da Goodfellow et al. [8] prevede l'esecuzione iniziale di k passi di aggiornamento del Discriminatore che dovrà

massimizzare la funzione

$$\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

seguiti da un passo di aggiornamento del Generatore che dovrà minimizzare la funzione

$$\mathbb{E}[\log(1 - D(G(z)))]$$

Dove $D(x) \in (0, 1)$ rappresenta la probabilità che x provenga da p_{data} , $G(z)$ è invece l'output prodotto dal generatore a partire dal rumore in input z . Le due funzioni possono essere dunque spiegate in maniera intuitiva partendo dai singoli componenti:

$$\log(D(x))$$

rappresenta il logaritmo della probabilità che il Discriminatore etichetti in maniera corretta i dati provenienti dal training set, mentre

$$\log(1 - D(G(z)))$$

rappresenta il logaritmo della probabilità che il Discriminatore assegni all'istanza $G(z)$ l'etichetta di "falso", dunque non proveniente da p_{data} . Il Discriminatore cercherà di massimizzare la probabilità che le sue predizioni su x e su $G(z)$ siano corrette, mentre il Generatore tenta di minimizzare la probabilità che il Discriminatore etichetti come "falsi" i propri output. Il logaritmo è applicato in entrambe le formule per le identiche motivazioni viste in precedenza durante la trattazione sulle funzioni di costo: la somma di probabilità fornisce dati migliori (rispetto al prodotto) per l'aggiornamento dei pesi [2].

Potrebbero manifestarsi casi in cui, durante la prima fase di addestramento, D possa etichettare con confidenza elevata le istanze generate da G , dunque $D(G(z)) \approx 0$ e di conseguenza $\log(1 - D(G(z))) \approx 0$, portando la funzione a saturazione. Per far fronte a questa dinamica si può optare per la sostituzione della funzione originaria di G con

$$\log(D(G(z)))$$

In questo caso, il generatore si troverà di fronte ad un problema di *massimizzazione*: la funzione rappresenta infatti la probabilità che D etichetti incorrettamente $G(z)$ come un dato proveniente dalla distribuzione p_{data} [8].

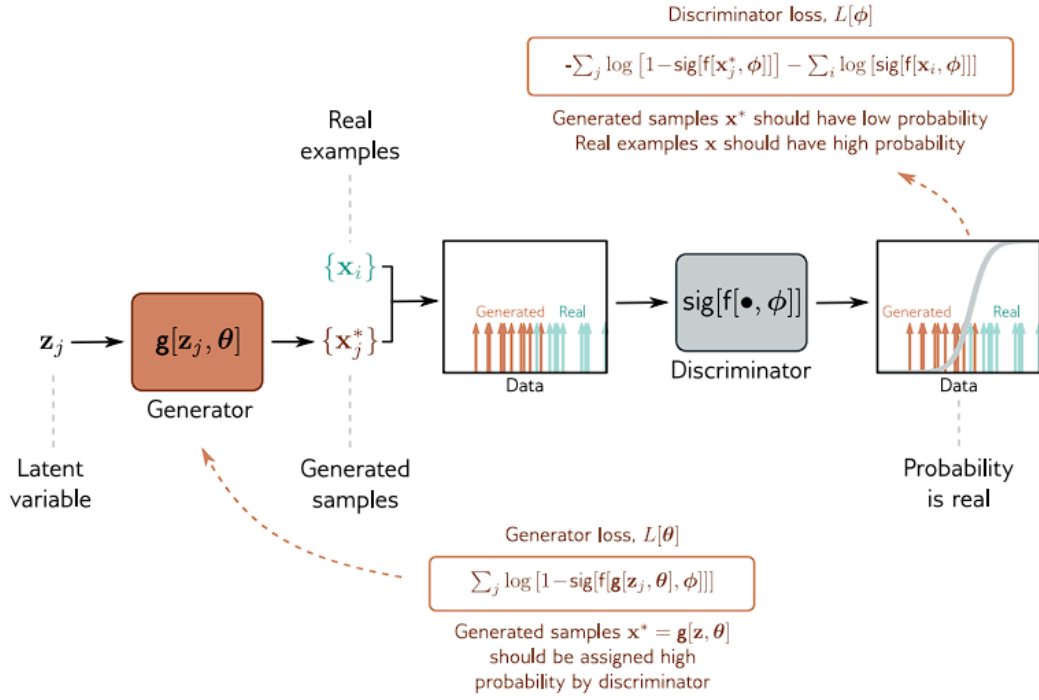


Figura 1.5: Esempio di addestramento di una GAN [2]

1.5.2 Stabilità delle GANs

La natura delle GANs può rivelarsi fatale e portare la rete a non stabilizzarsi correttamente, per questo motivo sono state introdotte una serie di soluzioni per intervenire in maniera efficace su questo aspetto. Salimans et al. [9] hanno presentato diverse tecniche per migliorare le prestazioni delle reti, come il *one-sided label smoothing*, che consiste, nel caso più generale, nel modificare i valori target del classificatore da 0 e 1 a .1 e .9, per evitare il fenomeno dell'overfitting. Nella soluzione proposta in [9], i valori target vengono fatti variare: da 0 a β e da 1 a α , il discriminatore sarà dunque descritto dalla seguente equazione:

$$D(x) = \frac{\alpha p_{data}(x) + \beta p_{model}(x)}{p_{data}(x) + p_{model}(x)}$$

Nei casi in cui $p_{data}(x) \approx 0$ e $p_{model}(x)$ fosse invece molto grande, D restituirebbe sempre β , spingendo G a non spostarsi verso i dati reali, la soluzione proposta è quella di imporre α per le classi positive, mentre per le negative si impone l'uso dello 0 [9]. Altre soluzioni invece, prevedono varianti del modello proposto originariamente da Goodfellow et al. [8]

1.5.3 Wasserstein Generative Adversarial Networks

Note anche con il nome di WGANs, furono presentate nel 2017 da Martin Arjovsky et al. [10] Questa variante delle GANs punta a risolvere una serie di problematiche presenti all'interno delle reti tradizionali, fra tutte, il bisogno di dover mantenere un equilibrio tra il training di D e di G e la necessità di ricorrere a specifiche scelte progettuali all'interno dell'architettura della rete [10]. La novità è rappresentata dalla distanza di Wasserstein (o "Earth-Mover distance"), il cui utilizzo permette di evitare il problema dell'annullamento del gradiente che insorge quando non vi è sovrapposizione tra la distribuzione reale e quella generata. La formula per questa distanza è:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$

Dove $\gamma(x, y)$ rappresenta la quantità di "massa" da spostare da x verso y per trasformare le distribuzioni $\mathbb{P}_r$ nella distribuzione $\mathbb{P}_g$ dunque, la distanza di Wasserstein è il "costo" del trasporto ottimale [10]. La Earth-Mover distance per due distribuzioni può essere inoltre riscritta nella forma di [10]:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \sup_{\|f\|_L \leq 1} \left\{ \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{z \sim p(z)}[f(G(z))] \right\}$$

Dove $f : \chi \rightarrow \mathbb{R}$ è una funzione 1-Lipschitz per cui vale la proprietà: $|f(x) - f(y)| \leq K \|x - y\|$ con $K = 1$. In [10], viene presentata una soluzione in cui f può essere sostituita con una funzione K-Lipschitz e perciò si considera una famiglia di funzioni $\{f_w\}_{w \in \mathcal{W}}$ tale per cui il problema può essere espresso nuovamente come:

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{z \sim p(z)}[f_w(g_\theta(z))]$$

In questa nuova formulazione il ruolo di "discriminatore" è assegnato alla rete f_w , detta anche *critic*, che a differenza delle normali D , che restituiscono un valore $D(x) \in (0, 1)$, questa restituirà in output un numero reale che misura la distanza tra la distribuzione reale $\mathbb{P}_r$ e la distribuzione generata $\mathbb{P}_g$; affinché si possa ricevere in uscita questo valore, in [10] vengono presentate reti in cui è rimosso l'ultimo strato sigmoidale. Per permettere a f_w di mantenere la proprietà di essere K-Lipschitz, in [10] viene esposto un metodo per approssimare questo vincolo tramite il *clipping* dei pesi. I pesi dovranno appartenere ad uno spazio compatto $\mathcal{W}$ e per garantire l'aderenza a questa condizione, i valori dovranno

essere "clampati" in un intervallo $[-c, c]^l$ ad ogni aggiornamento del gradiente, secondo la formula [10]:

$$w \leftarrow \text{clip}(w, -c, c)$$

Una volta stabilite le caratteristiche delle WGANs è possibile rivedere le scritture per il Discriminatore e per il Generatore. L'obiettivo di D diventa quello di *massimizzare* la stima empirica della distanza di Wasserstein:

$$\mathcal{L}_D = \mathbb{E}_{x \sim \mathbb{P}_r} [f_w(x)] - \mathbb{E}_{z \sim p_z} [f_w(G(z))].$$

Mentre G *minimizza* il valore medio che il critic assegna ai suoi campioni:

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p_z} [f_w(G(z))].$$

In questo modo D fornisce gradienti significativi anche quando le due distribuzioni non si sovrappongono, e G apprende a "spingere" p_g verso p_r , riducendo direttamente la distanza di Earth-Mover tra le due distribuzioni [10].

1.5.4 Two time-scale update rule

Il *Two time-scale update rule* (o TTUR) è una tecnica introdotta da Heusel et al. nel 2017 [11], di cui è stata dimostrata la convergenza (sotto una serie di assunzioni) ad un equilibrio di Nash locale ed è inoltre applicabile per qualunque variante delle GANs tradizionali [11]. Si considerano un D con vettore di parametri w ed un G con vettore di parametri θ di cui vengono definiti due gradienti stocastici:

$$\tilde{g}(\theta, w) = g(\theta, w) + M^w$$

$$\tilde{h}(\theta, w) = h(\theta, w) + M^\theta$$

dove g e h rappresentano i gradienti reali, e M delle variabili casuali, dunque $\tilde{g}$ e $\tilde{h}$ sono approssimazioni stocastiche dei gradienti reali [11]. Gli aggiornamenti dei pesi del Generatore e del Discriminatore avverranno secondo le formule:

$$w_{n+1} = w_n + b(n)(\tilde{g}(\theta, w))$$

$$\theta_{n+1} = \theta_n + a(n)(\tilde{h}(\theta, w))$$

dove $b(n)$ e $a(n)$ costituiscono rispettivamente i tassi di apprendimento per D e G . Come mostrato da Heusel et al.[11] questo approccio consente una serie di miglioramenti prestazionali durante l'apprendimento: se $b(n) \gg a(n)$, allora D apprenderà più rapidamente rispetto a G , consentendo al Discriminatore di stabilizzarsi prima di restituire feedback al Generatore. A livello empirico si è dimostrato più efficiente rispetto a tecniche tradizionali su dataset di immagini sia su GAN tradizionali che su WGAN [11].

1.5.5 Valutazione delle prestazioni

La natura generativa delle GANs permette una valutazione dei risultati sia qualitativa, attraverso un'analisi visiva dell'output prodotto, che quantitativa, attraverso l'uso di specifiche metriche. Come mostrato da Goodfellow et al., una metrica qualitativa può essere ottenuta chiedendo a un gruppo di soggetti umani di stabilire se l'immagine presentata sia per loro reale o generata. Queste valutazioni possono essere influenzate da diversi fattori come i feedback in tempo reale su eventuali errori di giudizio, che possono portare il soggetto verso una visione pessimistica dei dati a lui presentati [9].

1.5.6 Fréchet Inception Distance

Per ovviare alle problematiche dell'analisi qualitativa dei risultati prodotti dalle GANs sono state presentate una serie di metriche *quantitative* come la Fréchet Inception Distance, introdotta nell'ambito della valutazione delle Generative Adversarial Networks per la prima volta da Heusel et al. nel 2017 [11]. La misura determina la distanza tra le distribuzioni delle feature una volta estratte tramite reti *Inception* [11]. La distanza è calcolata tramite la formula:

$$\text{FID}(p_r, p_g) = \|\mu_r - \mu_g\|^2 + \text{Tr}(C_r + C_g - 2(C_r C_g)^{1/2})$$

dove

- $\mu_r, \mu_g \in \mathbb{R}^d$, sono i vettori di media delle due distribuzioni p_r, p_g ;
- $C_r, C_g \in \mathbb{R}^{d \times d}$ sono le matrici di covarianza;

- $(C_r, C_g)^{1/2}$ è la matrice quadrata del prodotto $C_r C_g$;

Nel contesto delle GANs, come affermato in precedenza, l'uso della FID è legato alla feature extraction da parte di reti specializzate (come la Inception) a partire dagli output generati [11]. E' immediato comprendere come una minore distanza di Fréchet indichi una maggiore qualità dei risultati prodotti.

Capitolo 2

Introduzione al Digital Signal Processing

2.1 Concetti Fondamentali

2.1.1 Digital Signal Processing

Il Digital Signal Processing (o "DSP") è una branca dell'informatica basata sull'analisi e la manipolazione di segnali analogici - siano essi ottenuti per mezzo di sensori, onde sonore o immagini - tramite tecniche ed algoritmi a seguito della loro conversione in segnali digitali [12]. La sua nascita coincide con la prima diffusione dei calcolatori elettronici ed il suo utilizzo trova tutt'ora grande impiego in vari ambiti: compressione di dati, generazione digitale di segnale audio, analisi di dati provenienti da strumenti di misurazione elettronici, analisi di immagini biomediche, ricerca nell'industria mineraria e ricostruzione della banda di segnali corrotti o di bassa qualità [12].

2.1.2 Campionamento e Quantizzazione

Campionamento e Quantizzazione sono due concetti cardine del processo di conversione "Analog-to-Digital" (o "ADC") in quanto i segnali sono descritti da funzioni continue, mentre qualsiasi strumento digitale manipola segnali discreti [12]. Il Campionamento è la misura del valore istantaneo di un segnale continuo a intervalli regolari di tempo che permette la trasformazione della variabile *indipendente* (spesso rappresentata dal tempo)

da continua a discreta. La corretta esecuzione del processo di campionamento è regolata dal *Teorema di Nyquist-Shannon* [13] (o "Teorema del campionamento") il quale stabilisce che, dato un segnale analogico $s(t)$ la cui banda di frequenze sia limitata dalla frequenza f_M e dato $n \in \mathbb{Z}$, il segnale $s(t)$ può essere univocamente ricostruito a partire dai suoi campioni $s(n\Delta t)$ presi a frequenza: $f_s = \frac{1}{\Delta t}$ se $f_s > 2f_M$, dove $\frac{f_s}{2}$ è nota come *frequenza di Nyquist*. Se la condizione non dovesse essere rispettata si potrebbe verificare il fenomeno dell'*aliasing* per cui le componenti ad alta frequenza del segnale vengono interpretate come frequenze più basse del dominio originale, rendendo impossibile la ricostruzione [12]. La Quantizzazione è invece il processo di conversione della variabile *dipendente* da continua a discreta, il valore campionato è approssimato al più vicino livello discreto disponibile e il numero di livelli totali dipende dalla risoluzione del DAC [12]. Se il DAC possiede una risoluzione di B bit e l'intervallo di uscita è pari a $[V_{min}, V_{max}]$ allora il numero di livelli disponibili è pari a:

$$L = 2^B$$

e l'ampiezza del passo (o LSB) sarà definito come:

$$\Delta_{DAC} = \frac{V_{max} - V_{min}}{2^B - 1} \approx \frac{V_{max} - V_{min}}{2^B}$$

Questa operazione introduce un "errore di quantizzazione" che può arrivare fino a $\pm \frac{1}{2}LSB$ ed è generalmente modellato come "rumore additivo uniforme" con media zero e deviazione standard pari a $\frac{LSB}{\sqrt{12}} \approx 0.29LSB$ [12]. In presenza di segnali a variazione lenta l'errore può manifestarsi come una distorsione sistematica e ciò consente una mitigazione tramite *dithering* ovvero l'aggiunta controllata di rumore per ottenere una rappresentazione più accurata del segnale originale [12].

2.1.3 Codifica nel Dominio del Tempo e della Frequenza

La codifica dell'informazione associata ad un segnale digitale può essere elaborata nel dominio del tempo e nel dominio della frequenza, in [14] è illustrato come i due metodi siano complementari ed entrambi concorrano ad ottenere informazioni dai segnali: la codifica nel dominio del tempo consente di ottenere maggiori informazioni sui ritardi e sulle fasi di transitorio mentre l'analisi in frequenza consente di comprendere il

comportamento spettrale del segnale. Nel dominio del tempo l'informazione è rappresentata come un segnale nel tempo perciò ogni campione ottenuto rappresenta il valore della funzione in un dato istante e ciò consente di estrarre informazioni da ognuno di questi, pertanto si può affermare che nella codifica nel dominio del tempo l'informazione risieda nella forma del segnale [12]. L'analisi del dominio del tempo consiste nella descrizione di un segnale $x[n]$ tramite la sua risposta ad un impulso $h[n]$ attraverso l'operazione di convoluzione della forma [14]:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

Dove l'output è la somma pesata dei singoli campioni passati attraverso il filtro $h[n]$.

Nel dominio della frequenza è la relazione armonica tra gruppi di campioni a fornire informazioni, ed in determinati ambiti, come quello musicale, le informazioni risiedono nel contenuto in frequenza piuttosto che nella forma d'onda del segnale [12]. Gli studi di Fourier hanno dimostrato come sia possibile decomporre qualsiasi funzione periodica continua (e per estensione anche un segnale) in una somma infinita di sinusoidi più semplici, ognuna delle quali è caratterizzata da frequenza, ampiezza e fase [12]. La rappresentazione in frequenza di $x[n]$ è definita da Fourier attraverso le sue Trasformate in due modalità a seconda del numero di campioni a disposizione. Per un numero infinito di campioni si ha [12]:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n}$$

dove ω è la frequenza angolare, periodica con periodo 2π .

L'applicazione di questa trasformata è spesso relegata agli ambiti teorici in quanto nelle applicazioni pratiche si avrà sempre un numero finito di campioni. $X[k]$ è dunque una somma finita di valori campionati e, se si assume che la sequenza sia definita per $n = 0, 1, \dots, N-1$ e sia nulla altrove, si può definire [12]:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1$$

dove si dimostra che per $w_k = \frac{2\pi}{N}k$ vale la seguente relazione:

$$X[k] = X(e^{j\omega_k}) \iff X(e^{j\omega})|_{\omega=\omega_k}$$

La ricostruzione della somma di campioni originale $x[n]$ è l'operazione inversa della Trasformata di Fourier vista in precedenza, ne segue che per $x[n]$ definita per tutti gli interi n , e se si conosce $X(e^{j\omega})$ allora si ha [12]:

$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

L'utilizzo di questa forma è vincolato agli aspetti teorici (come affermato in precedenza) oppure nei casi in cui è noto $X(e^{j\omega})$ in forma chiusa o numerica. Se invece si ha $X[k]$ allora la trasformata inversa per ottenere $x[n]$ è pari a [12]:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] w^{j\frac{2\pi}{N}kn}, n = 0, 1, \dots, N-1$$

Nel dominio della frequenza l'operazione di convoluzione diventa un prodotto della forma:

$$Y(e^{j\omega}) = H(e^{j\omega})X(e^{j\omega})$$

dove $H(e^{j\omega})$ è la funzione di trasferimento ottenuta a partire dalla trasformata di Fourier di $h[n]$:

$$H(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h[n] e^{-j\omega n}$$

In questa forma, la complessità del calcolo della convoluzione $O(n^2)$ è ampiamente ridotta, portando alla creazione di algoritmi efficienti per l'analisi di segnali anche molto grandi [12].

2.2 Rappresentazioni del segnale audio

La codifica nel tempo e nella frequenza definisce come un segnale viene trasformato per l'elaborazione mentre la rappresentazione definisce il formato e le strutture usati per archiviare i dati. Le varie tecniche di rappresentazione sono utilizzate sulla base di necessità come un uso parsimonioso della memoria o sulla fedeltà al segnale originale [12].

2.2.1 Pulse-Code Modulation Lineare

La Pulse-Code Modulation (o "PCM") è un metodo per rappresentare un segnale analogico in forma digitale campionandone periodicamente l'ampiezza e quantizzando su 2^B livelli

uniformemente distribuiti secondo la formula:

$$x_q[n] = \text{round}(x[n](2^{B-1} - 1)/(2^{B-1} - 1))$$

E' immediato comprendere che all'aumentare dei livelli aumenti sia la qualità della rappresentazione sia la banda richiesta, vi è dunque necessità di trovare un valore di compromesso a seconda del tipo di applicazione [12].

2.2.2 Comanding

Il Comanding (**COM**pression and **exPANDING**) è una tecnica che utilizza, a differenza di PCM, livelli di quantizzazione non lineari. Il principio è basato sulle caratteristiche dell'orecchio umano: il suono più potente che può tollerare è circa 120 dB che è un milione di volte l'ampiezza del suono più flebile che questo può percepire, cioè 0 dB, inoltre non può distinguere suoni che hanno una distanza minore di 1 dB [12]. Questo consente di stabilire una scala logaritmica di 120 possibili livelli di volume in un range di ampiezza pari a un milione [12].

Questa informazione consente una riduzione nel numero di bit necessari per rappresentare l'informazione: su livelli di quantizzazione uniformemente distribuiti sono necessari 12 bit per raggiungere la qualità di una telefonata, mentre su livelli non uniformi sono necessari solo 8 bit, consentendo inoltre un'aderenza migliore alle caratteristiche dell'udito umano. Sono due gli standard utilizzati per la compressione: μ -law e A-law, il primo è utilizzato nel Nord America e in Giappone e ha formula [12]:

$$y = \frac{\ln(1 + \mu x)}{\ln(1 + \mu)}$$

Dove $\mu = 255$ e per tanto la legge è anche nota come $\mu 255$ -law e vale per $0 \leq x \leq 1$.

La A-law è lo standard utilizzato nel resto del mondo ed ha formula [12]:

$$y = \begin{cases} \frac{1 + \ln(Ax)}{1 + \ln(A)}, & \text{per } \frac{1}{A} \leq x \leq 1 \\ \frac{Ax}{1 + \ln(A)}, & \text{per } 0 \leq x \leq \frac{1}{A} \end{cases}$$

Dove A ha valore pari 87.6.

Una volta compresso il segnale su questo viene applicata una conversione DAC. In fase di

ricostruzione si applica la legge inversa per riottenere il segnale nella sua forma originale, secondo le formule:

$$x = \operatorname{sgn}(y) \frac{(1 + \mu)^{|y|} - 1}{\mu}$$

$$x = \operatorname{sgn}(y) = \begin{cases} \frac{|y|(1 + \ln A)}{A}, & \text{per } |y| < \frac{1}{1 + \ln A} \\ \frac{e^{|y|(1 + \ln A)} - 1}{A}, & \text{altrimenti} \end{cases}$$

Questa tecnica consente una ricostruzione fedele del segnale compresso e permette una memorizzazione efficiente dei campioni senza elevate perdite in qualità [12].

2.3 La Short-Time Fourier Transform

La Short-Time Fourier Transform (o "STFT") è lo strumento più utilizzato nell'analisi di segnali non stazionari ed è un esempio di trasformata tempo-frequenza [15]. Il concetto può essere espresso tramite un esempio musicale: se si ascolta un brano musicale di un'ora contenente all'inizio il suono dei violini ed alla fine il suono dei tamburi e si analizza il contenuto tramite una semplice trasformata di Fourier, si notano i picchi in frequenza dovuti ai due tipi di strumento senza però avere alcuna indicazione sul *quando* questi vengano effettivamente suonati. Se successivamente si divide il brano in blocchi più piccoli, prima di cinque minuti e poi di un minuto, e li si analizza con Fourier si può localizzare l'esatto momento in cui i due strumenti vengono suonati [15]. Il ragionamento alla base della STFT è chiaro: suddividere un segnale di una lunghezza arbitraria in segmenti più piccoli ed analizzarli singolarmente per discernere le frequenze esistenti in quel particolare frangente di tempo. Il risultato dell'intera operazione sarà uno spettro che mostra l'evolvere delle frequenze nel tempo [15]. La qualità del risultato dipende dalla dimensione delle singole finestre da analizzare: per segnali troppo brevi otterremmo uno spettro senza alcuna correlazione con quello del segnale originale e questo è dovuto al fatto che, in questa trasformata, le proprietà del segnale sono combinate con quelle della *funzione di finestra*, cioè la funzione che determina come segmentare il segnale originale [15].

2.3.1 Rappresentazione matematica

Come anticipato in precedenza, l'analisi mediante STFT richiede l'introduzione di una *funzione di finestra* $w(t)$ tale che, se moltiplicata per un generico segnale $s(\tau)$, questa restituisce un segnale $s_t(\tau)$ tale per cui questo è *enfattizzato* in t e *soppresso* in tutti gli altri istanti di tempo, secondo la formula [15]:

$$s_t(\tau) = s(\tau)w(\tau - t)$$

Il segnale ottenuto è definito come segue:

$$s_t(\tau) = \begin{cases} s(\tau), & \text{se } \tau \text{ vicino a } t \\ 0, & \text{se } \tau \text{ distante da } t \end{cases}$$

Una volta descritto il risultato dell'operazione di moltiplicazione di un segnale per una funzione di finestra è possibile ottenere la formula per l'operazione di Short-Time Fourier Transform. Questa dipende dal tipo di segnale in input, per segnali continui la STFT è definita come:

$$STFT\{x(t, \omega)\} = \int_{-\infty}^{+\infty} x(\tau)w(\tau - t)e^{-j\omega t}d\tau$$

per segnali discreti, cioè campionati ad intervalli, la STFT è:

$$STFT\{x[m, k]\} = \sum_{n=-\infty}^{+\infty} x[n]w[n - m]e^{-j\frac{2\pi}{N}kn}$$

2.3.2 Funzione di finestra

Le funzioni di finestra hanno il compito di limitare l'evoluzione della sequenza, in quel frangente temporale, per rendere stazionarie le sue caratteristiche spettrali e permettere una migliore analisi [14]. La scelta della funzione di finestra gioca un ruolo cruciale nella qualità del risultato finale, in [15] è affermato che una buona localizzazione temporale richieda una finestra di dimensioni ridotte nel dominio del tempo mentre una migliore risoluzione della frequenza richieda finestre più larghe. Una caratteristica importante delle finestre risiede nelle loro trasformate: se si considera una finestra $w[n]$ di lunghezza L , il suo spettro $W(e^{j\omega})$ mostrerà un andamento oscillatorio in cui è possibile identificare un massimo, detto *lobo centrale*, che si manifesta attorno a $\omega = 0$ e dei *lobi laterali* con picchi minori a frequenze crescenti. Vi è quindi l'esigenza di trovare un compromesso per

ottenere un risultato di buona qualità e per fare ciò occorre prima esplorare le tipologie di funzioni di finestra utilizzate:

2.3.3 Finestra Rettangolare (o Finestra di Dirichlet)

Rappresenta la più semplice funzione di finestra, per l'analisi discreta è definita dalla funzione [16]:

$$w[n] = \begin{cases} 1, & 0 \leq n \leq N-1 \\ 0, & \text{altrimenti} \end{cases}$$

La sua DFT è pari a

$$W(\theta) = e^{-j\frac{N-1}{2}\theta} \frac{\sin[\frac{N}{2}\theta]}{\sin[\frac{1}{2}\theta]}$$

L'attenuazione tra il lobo centrale ed il primo lobo laterale è di circa 13 dB, inoltre possiede la massima risoluzione in frequenza [16].

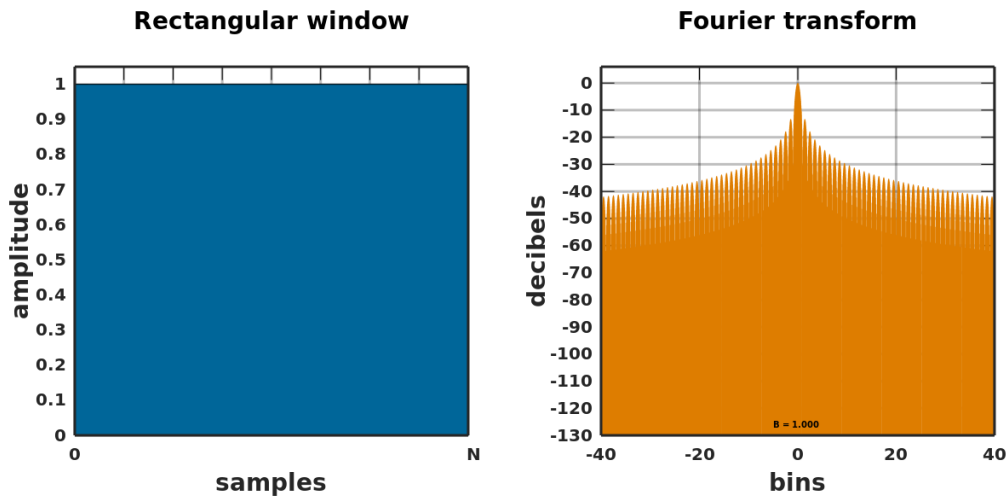


Figura 2.1: Finestra rettangolare e relativa trasformata

2.3.4 Finestra Triangolare (o Finestra di Fejer-Bartlett)

La finestra è definita dalla funzione [16]:

$$w[n] = \begin{cases} \frac{n}{N/2}, & \text{per } n = 0, 1, \dots, \frac{N}{2} \\ w[N-n], & \text{per } n = \frac{N}{2}, \dots, N-1 \end{cases}$$

La sua DFT è pari a:

$$W(\theta) = \frac{2}{N} \exp \left[-j \left(\frac{N}{2} - 1 \right) \theta \right] \left[\frac{\sin \left[\frac{N}{4} \theta \right]}{\sin \left[\frac{1}{2} \theta \right]} \right]^2$$

Il suo lobo centrale è il doppio di quello ottenuto da una finestra rettangolare e di conseguenza l'attenuazione con il primo lobo laterale è pari a 26 dB [16].

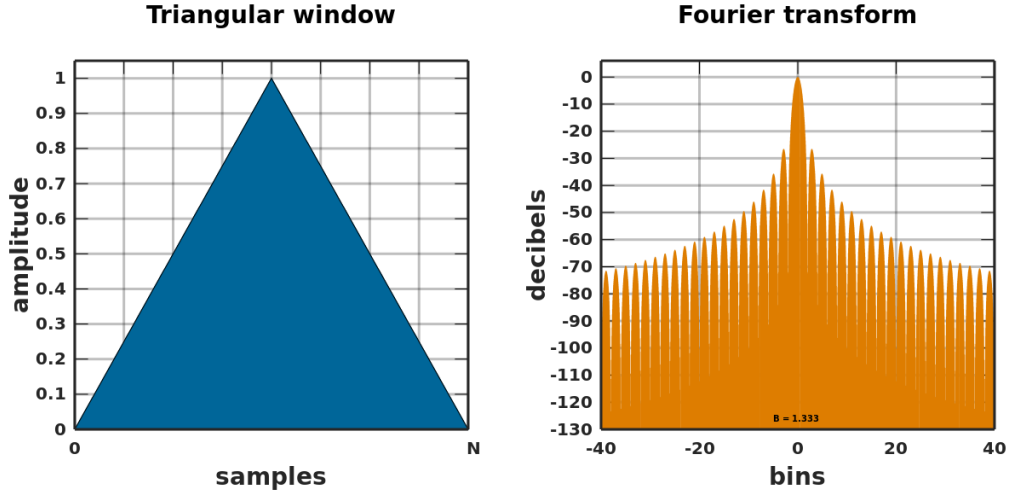


Figura 2.2: Finestra triangolare e relativa trasformata

2.3.5 Finestra $\cos^\alpha(X)$

Rappresenta una famiglia di finestre dipendenti dal parametro α che sfruttano le proprietà della trasformata del coseno [16], si ha:

$$w[n] = \cos^\alpha \left[\frac{n}{N} \pi \right]$$

con $-\frac{N}{2} \leq n \leq \frac{N}{2}$. La cui DFT è pari a:

$$w[n] = \sin^\alpha \left[\frac{n}{N} \pi \right]$$

con $0 \leq n \leq N - 1$.

Per $\alpha = 2$ si ottiene la *finestra di Hann* per cui si hanno rispettivamente [16]:

$$w[n] = \cos^2 \left[\frac{n}{N} \pi \right] = 0.5 \left[1.0 + \cos \left[\frac{2n}{N} \pi \right] \right]$$

e la trasformata:

$$w[n] = \sin^2 \left[\frac{n}{N} \pi \right] = 0.5 \left[1.0 - \cos \left[\frac{2n}{N} \pi \right] \right]$$

L'attenuazione tra il lobo centrale ed il primo lobo laterale è di circa 31 dB, ma il lobo centrale più largo implica una minore risoluzione in frequenza [16].

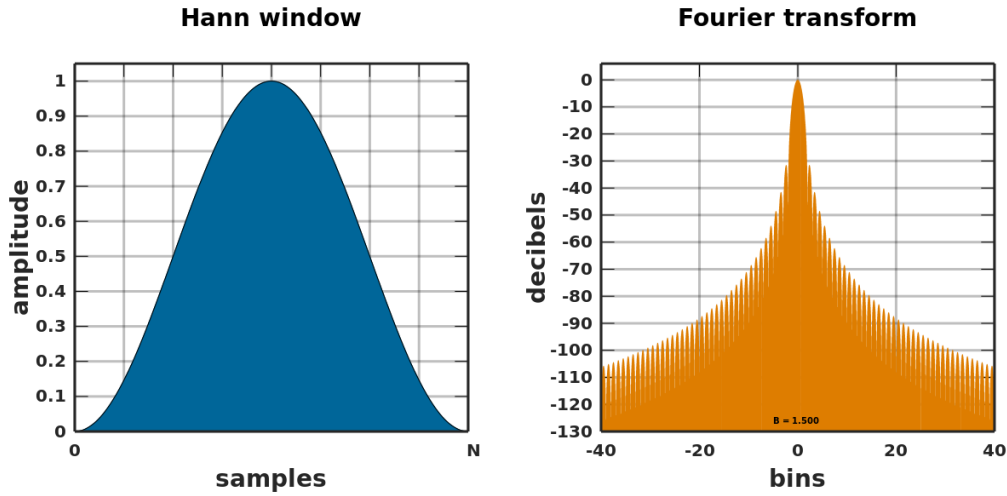


Figura 2.3: Finestra di Hann e relativa trasformata

2.3.6 Finestra di Hamming

La finestra di Hamming può essere pensata come una variazione della finestra di Hann della forma [16]:

$$w[n] = \alpha + (1 - \alpha) \cos\left[\frac{2\pi}{N}n\right]$$

Per determinati valori di α si ha una perfetta cancellazione del lobo centrale ottenendo un miglioramento nei livelli dei lobi laterali [16]. Selezionando $\alpha = \frac{25}{46} \approx 0.543...$ si ha la formula:

$$w[n] = 0.54 + 0.46 \cos\left[\frac{2\pi}{N}n\right]$$

con $-\frac{N}{2} \leq n \leq \frac{N}{2}$ e per la trasformata:

$$w[n] = 0.54 - 0.46 \cos\left[\frac{2\pi}{N}n\right]$$

con $0 \leq n \leq N - 1$.

L'attenuazione tra il lobo centrale ed il primo lobo laterale è di circa 43 dB ed inoltre questa finestra consente di raggiungere i livelli minimi dei lobi laterali [16].

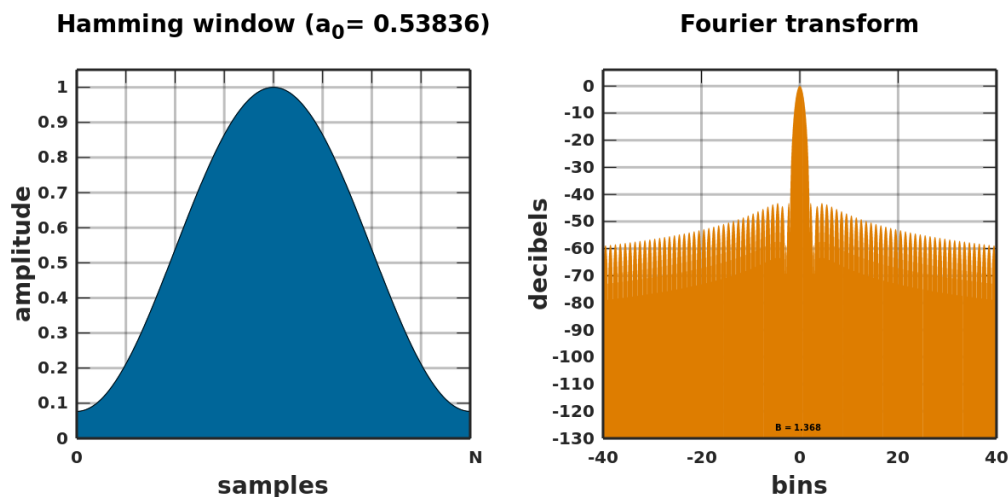


Figura 2.4: Finestra di Hamming e relativa trasformata

2.3.7 Mel-spectrogram

Un mel-spectrogram è una rappresentazione del contenuto spettrale di un segnale audio in cui l'energia è distribuita su una griglia di bande regolate secondo la scala mel [17], una scala percettiva che approssima la sensibilità dell'orecchio umano alle diverse frequenze. Rispetto a un normale spettrogramma (che utilizza scale lineari o logaritmiche di frequenza) il mel-spectrogram enfatizza le bande a bassa frequenza e comprime quelle ad alta frequenza, rispecchiando meglio la percezione uditiva umana [17]. La scala di Mel è una scala numerica psicofisica introdotta per la prima volta nel 1937 da Stevens et Al. in [18] per misurare la magnitudine *soggettiva* dell'altezza tonale. La scala è costruita in maniera sperimentale sulla base di feedback da parte dei partecipanti: ad ognuno dei soggetti sono stati presentati due suoni e veniva richiesto loro di correggere l'altezza del secondo fino a che questo non fosse percepito come *esattamente metà* del primo. L'intento era quello di valutare se i rapporti *percepiti* fossero gli stessi dei rapporti *numerici* noti [18]. Entrambi i suoni erano presentati con una intensità di 60 dB e non tutti i partecipanti sono riusciti con facilità a stabilire la *metà frequenza*, gli autori, infatti, osservano una media dei risultati pressoché identica tra gli auditori ad eccezione di uno, soprannominato "M", che si è discostato dal resto del gruppo nelle risposte [18]. Il mel è l'unità usata all'interno della scala ed il cui valore rispecchia la magnitudine percettiva dell'altezza tonale [18], nella scala si assegna il

valore di 1000 mels al tono di 1000 Hz presentato a 60 dB, da questa prima assunzione deriva l'intera scala che, come affermato in precedenza, si costruisce sulla base delle esperienze percettive dei partecipanti all'esperimento: metà del valore in mels corrisponderà ad un'altezza percepita come metà di quella originale [18]. Successivamente, la scala è stata migliorata ed è stata data una definizione formale dell'unità mel, in [17] questa è definita come:

$$m = 2595 \log_{10}(1 + f/700)$$

Per ottenere un mel-spectrogram vi è bisogno del calcolo dei *mel-frequency cepstral coefficients* e per ottenerli è necessario eseguire una serie di operazioni: il segnale analogico è suddiviso in brevi finestre temporali, con una sovrapposizione del 30-50%, a cui viene applicata una funzione di finestra sulle quali è infine eseguita un DFT allo scopo di ottenere $X(k)$ [17]. Ai segnali $X(k)$ sono applicate una serie finestre triangolari i cui centri seguono la scala di mel, il numero di finestre riportato in [17] è 20, poiché questo copre la banda della voce 0-4kHz, mentre in applicazioni più recenti si fa uso di un numero di finestre compreso tra 26 e 40, l'operazione assume dunque la forma di:

$$X_k = \sum_{i=1}^M H_k(i) |X(i)|^2$$

Questa produce un valore di energia di banda, pertanto le finestre hanno il compito di ricampionare lo spettro nelle bande percepite [17]. Una volta ottenuto X_k è possibile ottenere i coefficienti secondo la formula [17]:

$$c_n = \frac{1}{M} \sum_{k=1}^M \log X_k \cos\left(\frac{2\pi nk}{M}\right)$$

Il coefficiente c_0 è spesso scartato in quanto rappresenta l'energia media della finestra, c_1 è invece il bilanciamento energetico fra basse ed alte frequenze, i coefficienti c_i , $i > 1$ rappresentano un dettaglio spettrale sempre più fine [17]. Lo spettrogramma sarà dunque la matrice dei coefficienti c_n dove sull'asse x troviamo il tempo, sull'asse y la frequenza espressa in scala di mel e l'intensità dei coefficienti esprime l'energia percepita.

2.4 Tecniche di Ricostruzione

Dopo aver eseguito una scomposizione in finestre tramite STFT è indispensabile poter ricostruire il segnale originale in maniera fedele a partire dai campioni noti. Una sintesi deve evitare una serie di possibili problematiche che possono insorgere: aliasing, porzioni corrotte o mancanti oppure sovrapposizioni incontrollate. In [19] vengono analizzati due metodi di sintesi: il *Filter Bank Summation* e l'*Overlap Add* e la dualità delle due tecniche.

2.4.1 Filter Bank Summation

La Filter Bank Summation è un metodo che ricostruisce $x(n)$ modulando e sommando "banchi di filtri" [19]. Per ogni frequenza $\omega_k = \frac{2\pi k}{L}$, $X_n(e^{j\omega_k})$ è una rappresentazione passa-basso del segnale in una banda centrata in ω_k , l'idea è quindi quella di modulare $X_n(e^{j\omega_k})$ nella frequenza ω_k secondo la formula [19]:

$$y_k(n) = X_n(e^{j\omega_k})e^{j\omega_k n}$$

Il segnale $y(n)$ è dunque ricostruito a partire dalla somma dei singoli $y_k(n)$:

$$y(n) = \sum_k y_k(n) = \sum_k X_n(e^{j\omega_k})e^{j\omega_k n}$$

Con k pari al numero di frequenze utilizzate.

Una volta sostituita la definizione di:

$$X_n(e^{j\omega_k}) = \sum_{m=-\infty}^{\infty} w(n-m)x(m)e^{j\omega_k m}$$

ovvero la Short-Time Fourier Transform per un segnale $x(n)$, è possibile riscrivere $y(n)$ come:

$$y(n) = \sum_m w(n-m)x(m) \sum_k e^{j\omega_k(n-m)}$$

Se si assume che l'analisi sia eseguita in L frequenze uniformemente spaziate allora la sommatoria è equivalente a [19]:

$$y(n) = \sum_m w(n-m)x(m) \sum_{r=-\infty}^{\infty} L\delta(n-m-rL)$$

Dove $\delta(\cdot)$ è la funzione *Delta di Kronecker* discreta che per definizione vale:

$$\begin{cases} \delta(n) = 1, & n = 0 \\ \delta(n) = 0, & n \neq 0 \end{cases}$$

Si può dunque porre $m = n - rL$, ottenendo:

$$y(n) = L \sum_{r=-\infty}^{\infty} w(rL)x(n - rL)$$

Per $L \geq N$ la sommatoria può essere troncata fino a $r = 0$ ottenendo [19]:

$$y(n) = Lw(0)x(n)$$

Si è ottenuta una replica dell'input originale $x(n)$ scalata per un certo fattore $Lw(0)$ e scegliendo accuratamente i valori di L , purché valga $L \geq N$, è possibile ottenere una ricostruzione perfetta del segnale originale [19].

2.4.2 Overlap Addition

Un altro metodo per la sintesi si basa su una diversa interpretazione del segnale $X_n(e^{j\omega_k})$ che viene considerato come la trasformata di Fourier della sequenza [19]:

$$\hat{x}_n(m) = x(m)w(n - m)$$

Allora $x(m)$ può essere ricostruito a partire dalla trasformata di Fourier *inversa* di $X_n(e^{j\omega_k})$ divisa per la finestra (supponendo che abbia valore non-zero per tutti gli n considerati) [19]. Questa tecnica può portare ad una rappresentazione nel tempo *sotto-campionata* del segnale originale e risulta inefficace se vengono effettuate modifiche alla STFT. In [19] è presentato un approccio più robusto all'originale Overlap Addition per ottenere risultati migliori nella ricostruzione. La formula originale per la sintesi è la seguente:

$$y(n) = \sum_m \left[\sum_k X_m(e^{j\omega_k}) e^{j\omega_k n} \right]$$

Dove la sommatoria in m permette l'analisi in "sovrapposizione".

Per definizione si ha [19]:

$$y_m(n) = Lx(n)w(m - n)$$

Dove L è la dimensione della Trasformata di Fourier Discreta Inversa e dunque sommando i singoli $y_m(n)$ si ottiene [19]:

$$y(n) = \sum_m y_m(n) = Lx(n) \sum_m w(m - n)$$

Il termine $\sum_m w(m - n)$ è la somma della finestra traslata di m campioni [19]. Se la finestra è campionata ad una frequenza *sufficientemente densa* [19], allora vale:

$$\sum_m w(m - n) = W(e^{j0})$$

Dove $W(e^{j0})$ è la trasformata della finestra $w(n)$ valutata ad una frequenza $\omega = 0$.

È possibile ottenere la scrittura:

$$y(n) = Lx(n)W(e^{j0})$$

dimostrando che la formula originale dell'OLA permette la ricostruzione fedele di $x(n)$ attraverso la sovrapposizione delle sezioni delle forme d'onda ma si è dipendenti dalla frequenza di campionamento [19]. Nello specifico, all'aumentare del tasso di campionamento è possibile ridurre la presenza di errori di aliasing [19].

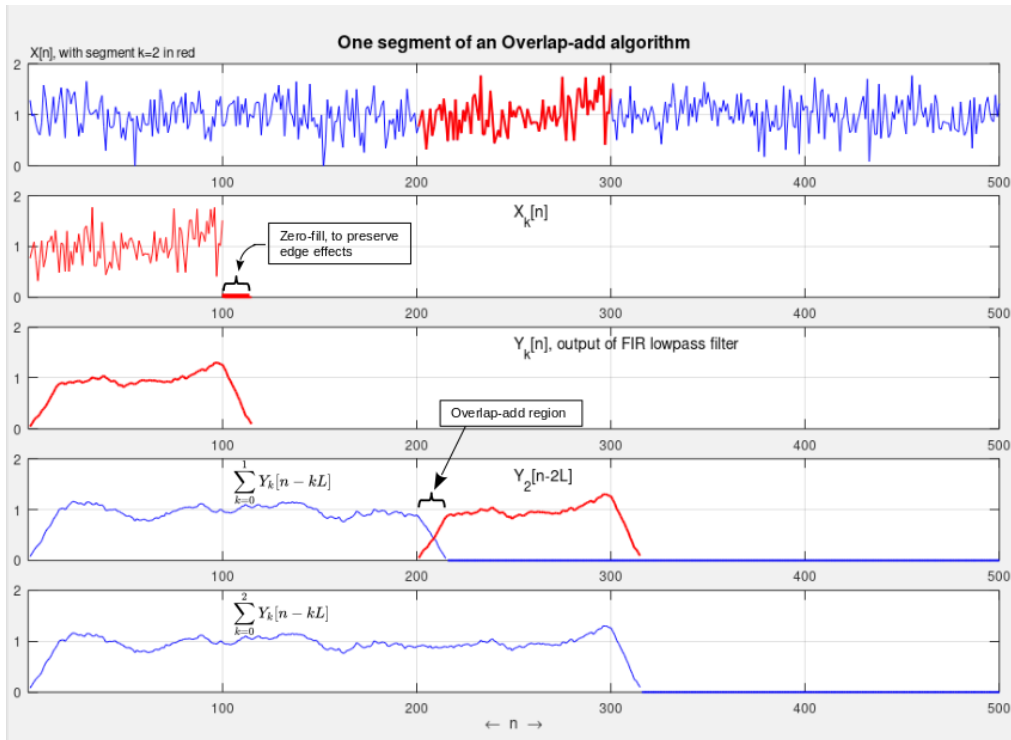


Figura 2.5: Visualizzazione della tecnica Overlap Add

2.4.3 Frequenze di campionamento

I metodi di sintesi illustrati in precedenza risultano estremamente dipendenti dalle frequenze di campionamento ed è pertanto necessario stabilire in maniera rigorosa le frequenze per l'ottenimento di una ricostruzione perfetta del segnale originale.

2.4.4 Campionamento nel tempo

Per una data frequenza ω_k , $X_n(e^{j\omega_k})$ equivale all'operazione di filtraggio del segnale $x(n)e^{-j\omega_k n}$ con la finestra $w(n)$ [19]. Ne segue che se $W(e^{j\omega})$ ha larghezza di banda B allora per il Teorema di Nyquist-Shannon [13] $X_n(e^{j\omega_k n})$ deve essere campionato nel tempo con un passo pari a [19]:

$$\Delta t \leq \frac{1}{2B}$$

Nel dominio discreto, dato un segnale $x(n)$ campionato ad una frequenza F_s e data una finestra $w(n)$ di lunghezza N , l'ampiezza di banda sarà pari a [19]:

$$B \approx \frac{2F_s}{N}$$

La Short-Time Fourier Transform dovrà essere calcolata a partire da un numero di campioni pari a:

$$NT = N \frac{4F_s}{N} = 4F_s$$

2.4.5 Campionamento in frequenza

Per ottenere il tasso di campionamento nel dominio della frequenza è necessario partire dalla Trasformata di Fourier di $X_n(e^{j\omega_k})$: poiché la trasformata inversa è "tempo-limitata" si può utilizzare il Teorema del Campionamento [13] per campionare $X_n(e^{j\omega_k})$ ad un tasso pari ad almeno il doppio della durata totale della finestra [19]. Siccome la trasformata inversa è pari a $x(m)w(n-m)$ e il segnale è formato da N campioni, per una ricostruzione perfetta è necessario che il segnale sia campionato nelle frequenze [19]:

$$\omega_k = \frac{2\pi k}{N}$$

Con $k = 0, 1, \dots, N-1$.

2.4.6 Dualità tra FBS e OLA

I due metodi illustrati sono utilizzati per la sintesi di un segnale moltiplicato per un fattore di scala arbitrario ed in [19] è presentata la dualità delle tecniche sotto forma di due relazioni:

$$\sum_m w(n - m) = W(e^{j0})$$

Questa è la base del metodo OLA che consiste nella somma delle trasformate inverse che restituisce $W(e^{j0})$ e che, se normalizzato a 1, garantisce la ricostruzione perfetta del segnale.

$$\frac{1}{L} \sum_k W(e^{j(\omega - \omega_k)}) = w(0)$$

Dove $\omega_k = \frac{2\pi k}{L}$ e rappresenta l'idea di fondo di FBS: se L è sufficientemente grande da campionare ad una frequenza di almeno $2B$ allora la somma restituisce $w(0)$, consentendo una ricostruzione perfetta. In [19] si è dimostrato come i due metodi siano derivabili l'uno a partire dall'altro, con l'opportuna trasformazione della variabile *dipendente* nella sua trasformata di Fourier corrispondente e intercambiando i ruoli di tempo e frequenza.

Capitolo 3

Audio Super Resolution

L'Audio Super Resolution (o "ASR") è una tecnica nel campo dell'elaborazione del segnale audio che mira a ricostruire componenti ad alta frequenza mancanti da segnali audio a bassa risoluzione. Questa tecnologia trova applicazione in diversi contesti, dalla telefonia alla restaurazione di registrazioni storiche, fino al miglioramento della qualità audio in sistemi di sintesi vocale e musicale. L'obiettivo fondamentale è quello di estendere la larghezza di banda di un segnale audio degradato o limitato, ripristinando le informazioni perse durante processi di campionamento a bassa frequenza, compressione o deterioramento nel tempo. I primi approcci erano basati su tecniche di rettificazione ed appiattimento dei segnali [20] mentre ora sono impiegate moderne tecniche di apprendimento e di generazione. I primi risultati di elevata qualità ottenuti utilizzando una rete neurale per l'Audio Super Resolution si devono a Kuleshov et al. nel 2017 [21], i quali creano un modello convoluzionale con l'obiettivo di migliorare la qualità dei segnali audio contenenti solo frequenze a banda bassa, spesso ottenuti a partire da strumenti di rilevamento di bassa qualità oppure datati (come nel caso di vecchie registrazioni musicali) [22]. Nel contesto dei segnali audio, la qualità percepita aumenta sensibilmente quando sono presenti sia tutte le frequenze dello spettro dell'udibile sia le armoniche [22] e ciò consente di avere una maggiore fedeltà del segnale ricostruito rispetto all'audio originale.

Il processo si basa sulla stessa logica utilizzata nella Image Super-Resolution ed ha mostrato sperimentalmente un notevole miglioramento rispetto a tecniche di super risoluzione basate su sovra-campionamento del segnale originale [21]. Nelle tecniche di

super-risoluzione precedenti, l'audio era utilizzato in forma grezza consentendo una maggiore flessibilità a discapito della complessità computazionale su cui grava l'elevato numero di campioni necessari. In [21] la soluzione proposta permette una super-risoluzione a partire da un piccolo frammento pari a circa il 15-20% dell'audio originale, riducendo notevolmente la complessità computazionale.

3.1 Definizione del problema

Il problema dell'Audio Super Resolution può essere formalmente definito come segue: dato un segnale audio a bassa risoluzione $x_l = \{x_{1/R_l}, x_{2/R_l}, \dots, x_{R_l T/R_l}\}$ campionato a frequenza R_l , l'obiettivo è sintetizzare un segnale ad alta risoluzione $x_h = \{x_{1/R_h}, x_{2/R_h}, \dots, x_{R_h T/R_h}\}$ con frequenza di campionamento $R_h > R_l$, dove T rappresenta la durata del segnale in secondi [21]. Il rapporto di sovra-campionamento $r = R_h/R_l$ definisce il fattore di aumento della risoluzione. Tipicamente si utilizzano valori di $r \in \{2, 4, 6, 8\}$, con $r = 4$ che rappresenta un compromesso tra qualità della ricostruzione e complessità computazionale. Ad esempio, un segnale telefonico campionato a 8 kHz può essere portato alla qualità di un CD (44.1 kHz) con un fattore di sovra-campionamento di circa 5.5. L'ASR si riconduce ad un mapping "one-to-many" poiché molteplici possibili campioni audio ad alta risoluzione corrispondono a un dato campione a bassa risoluzione, in quest'ottica il problema è considerato "mal posto" [23]. In letteratura sono state proposte diverse soluzioni al problema, molte di queste facenti uso delle GANs che, nonostante abbiano ottenuto risultati di elevata qualità, sono note per la complessità del loro addestramento e per la presenza di artefatti nell'output. Sono diverse le architetture proposte per l'Audio Super Resolution, ognuna con determinate caratteristiche ed approcci al problema che le rendono diverse le une dalle altre.

Di seguito sono esaminati alcuni dei principali modelli:

3.2 Modelli proposti

3.2.1 Architettura di Kuleshov et al.

La rete neurale proposta da Kuleshov et al. in [21] rappresenta una pietra miliare tra le soluzioni al problema dell'ASR in quanto è il primo modello basato sul deep learning ad aver mostrato risultati significativi nella ricostruzione nel dominio del tempo.

Inizialmente il segnale grezzo passato in input viene proiettato in uno spazio ad alta dimensionalità mediante l'interpolazione cubica, seguono poi B blocchi per il sotto-campionamento, ognuno dei quali esegue le seguenti operazioni:

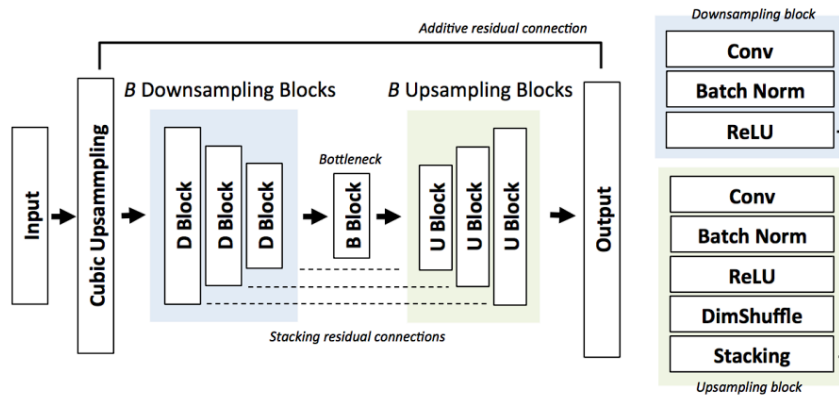


Figura 3.1: Architettura della rete proposta da Kuleshov et al. [21], con dettaglio dei blocchi di sovra-campionamento

1. Convoluzione con $stride=2$ (per ridurre la dimensionalità) unidimensionale per l'estrazione delle feature locali;
2. Normalizzazione per stabilizzare l'addestramento;
3. Applicazione della funzione di attivazione ReLU;
4. Incremento del numero di filtri ad ogni blocco.

A seguire si trovano altri B blocchi disposti in maniera speculare ai precedenti, in questo caso impiegati per il sovra-campionamento. Vengono stabilite delle *skip connections* per concatenare l' i -esimo tensore di sotto-campionamento con il $(B-i)$ -esimo tensore di

sovra-campionamento permettendo di utilizzare nella fase di ricostruzione le feature estratte in precedenza [21]. Segue un layer "Subpixel" unidimensionale che riorganizza i canali risultando più efficiente della tradizionale deconvoluzione. Viene in ultimo inserita una connessione residuale tra l'output dell'interpolazione cubica e l'output finale, in questo modo il modello dovrà solo imparare a migliorare l'approssimazione cubica iniziale, facilitando l'addestramento e la convergenza [21].

Il modello ha mostrato sperimentalmente una elevata qualità nella ricostruzione assieme ad una serie di problematiche, come l'introduzione di rumore di fondo o di artefatti udibili quando si tenta un sovra-campionamento elevato oppure performance non ottimali su dataset molto vari [21].

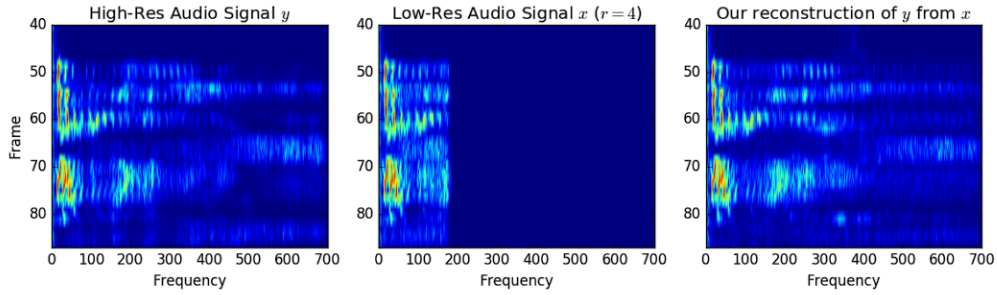


Figura 3.2: A sinistra lo spettrogramma completo della traccia, al centro l'audio sotto-campionato ottenendo con perdita delle alte frequenze, a destra lo spettrogramma generato dalla rete in cui è possibile notare la presenza di componenti frequenziali non previste

3.2.2 Architettura WSRGlow

La rete neurale WSRGlow, introdotta da Zhang et al. [23] si pone come una novità nel panorama dell'ASR utilizzando un diverso approccio sia rispetto ai modelli basati su Reti Neurali Convoluzionali, che spesso restituiscono risultati non soddisfacenti, sia rispetto a modelli facenti uso di GANs, ritenute difficili da addestrare e suscettibili alla generazione di artefatti [23]. Il principio alla base della tecnica è il "*normalizing flow*" che modella una trasformazione biiettiva tra la distribuzione semplice z (una Gaussiana standard):

$$z \sim \mathcal{N}(z; 0, \mathbf{I})$$

e la distribuzione x che modella lo spazio dell'audio [23]. L'architettura della rete si basa su una serie di layer di *Glow* nei quali un campione z è estratto da una distribuzione normale e successivamente attraversa una serie di layer invertibili f_i che trasformano la distribuzione originale in quella desiderata secondo la formula:

$$x = f_0 \circ f_1 \circ \dots \circ f_k(z)$$

Durante la fase di addestramento l'audio in alta risoluzione è considerato un vettore e il tutto viene compresso in gruppi da 8 elementi. Il vettore passa dunque attraverso i layer invertibili per essere trasformato nella distribuzione semplice e proprio la caratteristica dell'invertibilità consente di poter calcolare la log-verosimiglianza attraverso un mapping invertito secondo la formula [23]:

$$\log p_\theta(x) = \log p_\theta(z) + \sum_{i=1}^k \log | \det(\mathbf{J}(f_i^{-1}(x))) |$$

con

$$z = f_k^{-1} \circ f_{k-1}^{-1} \circ \dots \circ f_0^{-1}(x)$$

cioè la trasformazione della distribuzione dello spazio dell'audio x nella distribuzione Gaussiana.

$\mathbf{J}$ è la matrice di derivate parziali della funzione f_i della forma:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

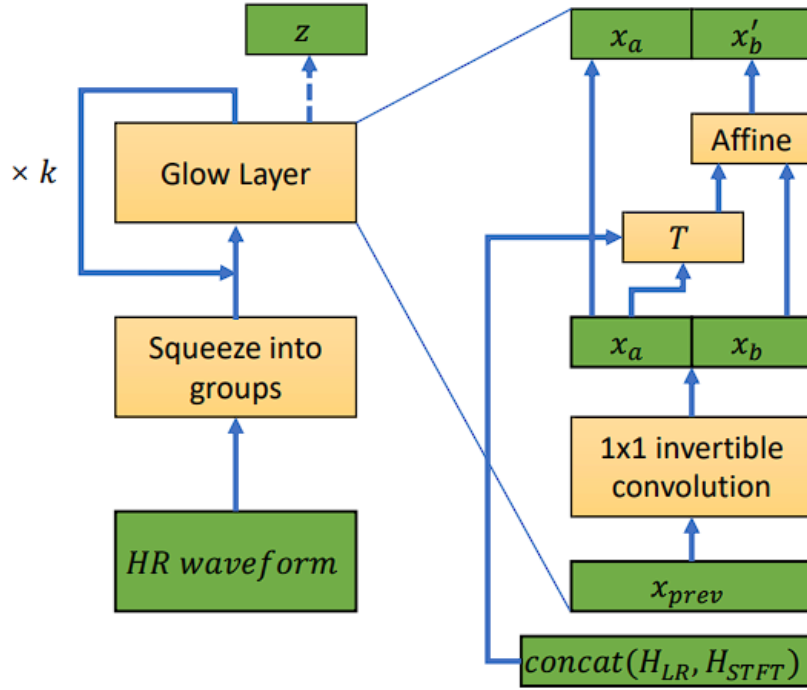


Figura 3.3: Architettura della rete WSRGlow [23], durante l’addestramento il modello segue la direzione delle frecce, durante l’inferenza segue il verso opposto, con dettaglio dell’architettura del layer di Glow

Ogni strato della Glow è composto da una convoluzione 1x1 invertibile f_{conv} :

$$f_{conv}^{-1} = W \cdot x$$

dove W è la matrice di convoluzione e un layer di *affine coupling* $f_{coupling}$ che permette di garantire l’invertibilità dello strato [23]. L’operazione di *affine coupling* si articola in diverse fasi: i canali del tensore x in input vengono separati in due metà, un metà dei canali sarà passata in input ad una trasformazione T che produce un termine additivo t , dall’altra metà si otterrà invece un termine moltiplicativo s [23]:

Divisione dei canali in due metà

$$x_a, x_b = split(x)$$

Applicazione della trasformazione T

$$(\log s, t) = T(x_a, concat(H_{LR}, H_{STFT}))$$

Nello specifico

$$\text{concat}(H_{LR}, H_{STFT})$$

rappresenta una concatenazione degli encoding in bassa risoluzione del segnale audio, differenziando così WSRGlow dalle architetture WaveNet che invece utilizzano un mel-spectrogram come input per la trasformazione T .

H_{LR} è l'encoding a bassa risoluzione ottenuto applicando una compressione $\mu - law$ e quantizzando successivamente i campioni ottenuti su 256 possibili valori. Si creano poi dei vettori di embedding per ognuno dei valori e infine si raggruppano in blocchi di campioni di un numero adeguato per soddisfare la dimensionalità di T [23].

H_{STFT} è l'encoding delle informazioni presenti nel dominio della frequenza. La novità risiede nell'utilizzo sia dello spettrogramma della magnitudine che dello spettrogramma della fase, una tecnica che si è dimostrata utile in diverse applicazioni [23].

Lo spettrogramma della magnitudine si ottiene applicando la STFT al segnale ed estraendone la parte reale:

$$S(x) = \text{real}(STFT(x))$$

Lo spettrogramma della fase si ottiene invece estraendo la parte immaginaria dalla $STFT(x)$ ed applicando successivamente quantizzazione ed embedding:

$$E_\phi = \text{Embedding}(\text{Quantize}(\text{imag}(STFT(x))))$$

In ultimo si concatenano i due risultati:

$$H_{STFT} = \text{concat}(S(x), E_\phi(x))$$

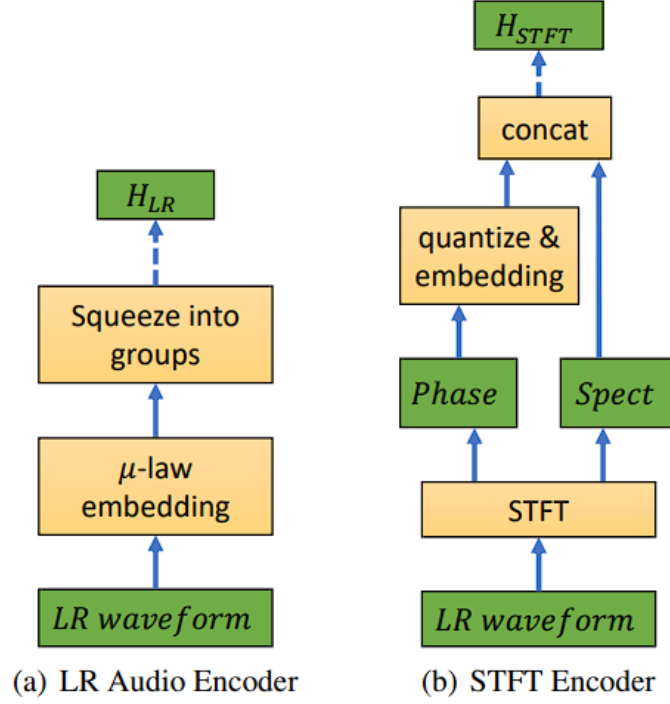


Figura 3.4: Struttura degli Encoder presenti nella rete WSRGlow [23], a sinistra l'encoder a bassa risoluzione, a destra l'encoder del dominio della frequenza

Segue il prodotto elemento-per-elemento tra x_b e s , che agisce come una matrice di scaling, il tutto sommato con il vettore di traslazione t :

$$x'_b = s \odot x_b + t$$

l'output del layer sarà dunque la concatenazione di x_a e x'_b :

$$f_{coupling}^{-1}(x) = \text{concat}(x_a, x'_b)$$

In questo caso la trasformazione T non deve essere necessariamente invertibile perché saranno gli strati di *affine coupling* a garantire la proprietà alla rete [23].

Sperimentalmente è stato mostrato come l'utilizzo di WSRGlow porti una serie di miglioramenti rispetto all'utilizzo delle GANs come il training più stabile, la convergenza veloce e dei migliori risultati sia in termini di metriche oggettive che soggettive; l'approccio è però più lento rispetto alle reti convoluzionali e porta con sé un utilizzo maggiore della memoria [23].

3.2.3 Architettura AERO

La rete neurale AERO, introdotta da Mandel et al. [24], presenta un approccio innovativo all'ASR operando interamente sul dominio spettrale e predicendo l'intero range di frequenze anziché concatenare bande esistenti e generate. La scelta di non utilizzare il dominio temporale lo si deve in larga parte al costo computazionale dell'estrazione dei campioni. Dato un segnale in bassa risoluzione $x \in \mathbb{R}^{T/s}$ ottenuto sotto-campionando un segnale in alta risoluzione $y \in \mathbb{R}^T$ dove s è un fattore di scaling fissato, l'obiettivo del modello è generare $\hat{y} \approx y$ [24].

Lavorando nel dominio della frequenza, al segnale x è inizialmente applicata una STFT con f bande di frequenza, *hop-length* pari a $\frac{f}{ks}$ ed una finestra di lunghezza $\frac{f}{s}$, per ottenere X . Il modello è dunque addestrato per predire lo spettrogramma delle alte frequenze, l'output $\hat{Y}$ è infine riconvertito in segnale attraverso una Inverse-STFT. Il modello opera su spettrogrammi a valori complessi e pertanto la parte reale e quella immaginaria sono concatenati come canali separati così da gestire correttamente le informazioni su magnitudine e fase ed evita il problema, presenti in altri modelli, della ricostruzione separata della fase [24].

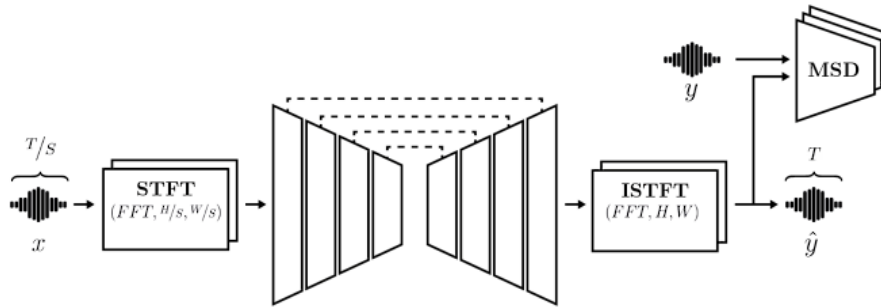


Figura 3.5: Architettura della rete AERO [24], il segnale è convertito inizialmente nel proprio spettrogramma tramite STFT; dopo aver attraversato i blocchi della rete questo viene riconvertito nuovamente in segnale. Durante l'addestramento vengono impiegati un insieme di "Discriminatori multi-scala"

Il modello segue l'architettura U-Net con 4 strati di encoding e di decoding, i layer corrispondenti sono tra loro concatenati da *skip connections*. I livelli di encoding

prendono in input lo spettrogramma del segnale, il quale viene elaborato inizialmente da un *Frequency Transformer Block* che applica meccanismi di attenzione per catturare le dipendenze e le relazioni spettrali; si applica poi una convoluzione unidimensionale sull'asse delle frequenze, seguono successivamente due rami residuali per la compressione, riducendo di circa il 50% i canali intermedi, ai quali viene applicata una funziona di attivazione "Snake":

$$Snake(x) = x + \frac{1}{\alpha} \sin^2(\alpha x)$$

la quale migliora l'apprendimento delle funzioni periodiche. L'output, composto dalla somma dei due rami e dell'input, viene successivamente sotto-campionato; il fattore di campionamento segue lo schema [4, 4, 2, 2] ottenendo, in output all'ultimo livello di encoding, un vettore compresso complessivamente 64 volte. Negli strati interni dell'encoder (il terzo ed il quarto) sono presenti moduli LSTM per cogliere le dipendenze temporali e meccanismi di attenzione per le relazioni a lungo raggio, permettendo così al modello di tenere in considerazione sia il contesto temporale che spettrale durante la ricostruzione [24].

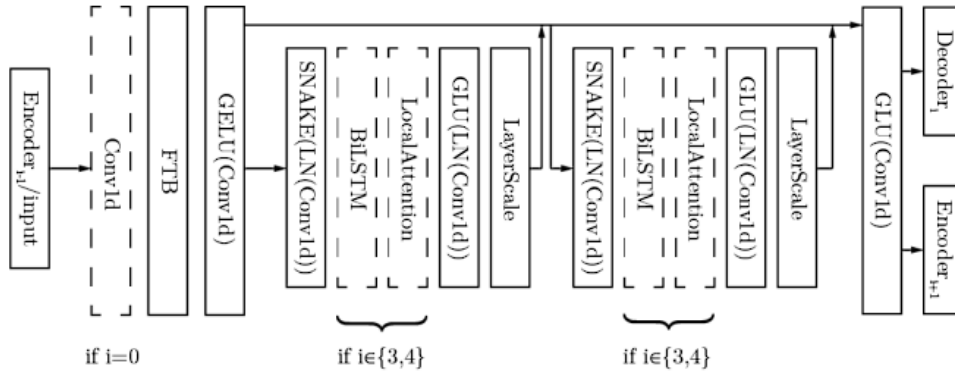


Fig. 2: Encoder layer.

Figura 3.6: Struttura dell'encoder della rete AERO [24]

Gli strati di decoding seguono una struttura speculare ed hanno il compito di trasformare il vettore in input in uno spettrogramma della stessa dimensione di quello passato in input all'encoder. Sperimentalmente si è dimostrato come l'approccio innovativo di

AERO (cioè la predizione dell'intero range di frequenze) produca un minor numero di artefatti che, in altri modelli, possono sorgere dalla "giunzione" tra la banda disponibile e quella ricostruita [24].

3.2.4 Architettura ViT-SR

Le reti ViT-SR, introdotte da Nisticò et al. [22], presentano un approccio innovativo nel campo dell'Audio Super Resolution introducendo l'utilizzo dei Vision Transformer nelle architetture basate su Generative Adversarial Networks. Il Generatore ha il compito di ricostruire le frequenze mancanti mentre il Discriminatore ha il compito di stabilire se i dati passati in input sono reali o meno.

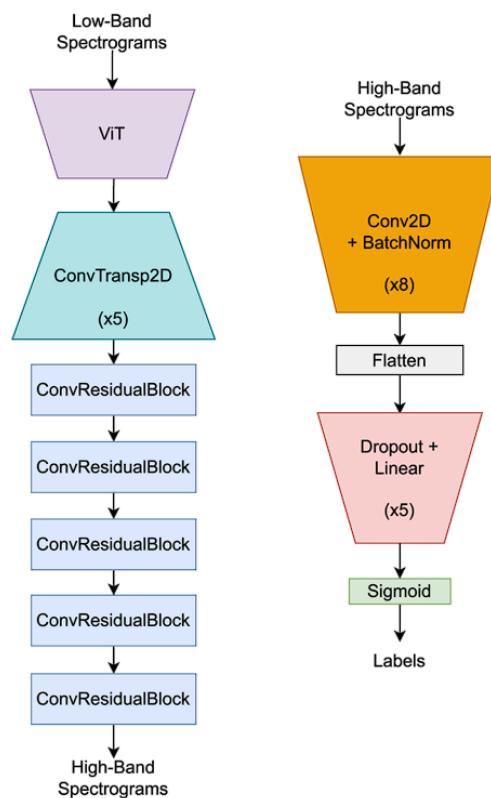


Figura 3.7: Architettura della rete ViT-SR [22], a sinistra la struttura del Generatore, a destra il Discriminatore

Il primo modulo del Generatore è un Vision Transformer che lavora come un encoder: viene fornito in input un'immagine quadrata dello spettrogramma e a sua volta sarà divisa

in patch quadrate di dimensioni minori [22]. Ognuna di queste viene linearizzata in un vettore unidimensionale sul quale è successivamente applicata una proiezione lineare per ottenere un embedding di dimensione fissata. Il Transformer possiede è composto da una serie di layer [22]:

- Layer di normalizzazione, posto prima e dopo il blocco di Multi-Head Attention;
- Blocco di Multi-Head Attention, per implementare il meccanismo di attenzione, permettendo così al modello di focalizzarsi su diversi parti dello spettrogramma;
- Connessioni residuali poste dopo il blocco di attenzione;

Nell'architettura originale del Vision Transformer è presente un "Multi-Layer Perceptron" che permette di usare il modulo come un classificatore, in ViT-SR questo strato è rimosso, trasformando il Transformer in un encoder. A seguire si trovano i 5 layer di deconvoluzione (ConvTransp2D), che hanno il compito di espandere progressivamente la rappresentazione (che comprende sia frequenze reali che generate), fino alla dimensione desiderata. Seguono poi 5 "ConvResidualBlock" che, tramite *skip connections* poste strategicamente ogni 2 layer, hanno il compito di rifinire i dati prima di essere restituiti in output [22]. La funzione di costo utilizzata per il Generatore è la *Mean Squared Error* che è adatta alla quantificazione dell'errore di ricostruzione.

Il Discriminatore è composto da 6 layer di convoluzione bidimensionale con funzione di attivazione GeLU per ridurre la dimensione dell'immagine; un layer di appiattimento per la linearizzazione delle feature e una serie di layer lineari con un numero di neuroni decrescente. L'ultimo strato è composto da una funzione sigmoide che darà in output la predizione sulla natura dei dati in input [22]. La funzione di loss per il Discriminatore è la *Binary Cross-Entropy* la quale è indicata per task di classificazione. Questa architettura raggiunge risultati migliori rispetto ad altri modelli (AERO) per quanto riguarda la Log-Spectral Distance, e mostra ottimi risultati nella ricostruzione delle armoniche che, come già affermato in precedenza, contribuiscono sensibilmente a migliorare la qualità percepita.

3.3 Metriche di valutazione per Audio Super Resolution

Nell'ambito dell'Audio Super Resolution, o più in generale nell'ambito della generazione, l'output di un modello può essere valutato tramite metriche soggettive (in questo caso si pensi all'ascolto della traccia) che, come già trattato in precedenza durante l'analisi delle GANs, possono essere soggette ad errore o a possibili bias. Per poter avere metriche oggettive della qualità del prodotto generato vengono impiegati nel contesto dell'ASR, il *Signal-to-Noise ratio* (o "SNR") e la *Log-spectral Distance* (o "LSD") [21] [22] [23] [24] [25], entrambe sono usate per confrontare spettrogrammi con la stessa forma.

3.3.1 Signal-to-Noise ratio

Il Signal-to-Noise ratio è il rapporto tra la potenza di un segnale e la potenza del rumore (quando questo è modellato come rumore Gaussiano) nella registrazione [26], la sua formula è la seguente:

$$SNR = \frac{P_s}{P_N}, 0 \leq SNR \leq \infty$$

Dove P_s è la potenza del segnale, calcolata come:

$$P_s = \frac{1}{T} \int_0^T s^2(t) dt$$

$s(t)$ rappresenta il segnale mentre T è la durata dell'intervallo di osservazione; per segnali periodici T equivale al loro periodo.

Nel caso di processi stocastici la potenza è definita da una funzione di correlazione $R_s(\tau)$ valutata all'origine:

$$R_s(\tau) = \mathbb{E}[s(t)s(t + \tau)]$$

$$P_s = R_s(0)$$

P_N è invece la potenza del rumore identificato da $N(t)$ calcolata come:

$$P_N = R_N(0)$$

Spesso si tende ad esprimere la misura del Signal-to-Noise ratio in decibel e per ottenerla si utilizza la formula:

$$SNR(dB) = 10 \log_{10} \frac{P_s}{P_N}$$

3.3.2 Log-spectral Distance

La Log-Spectral Distance è utilizzata per valutare la qualità delle ricostruzioni delle singole frequenze [23], la sua formula è la seguente [27]:

$$LSD(x, y) = \frac{1}{L} \sum_{l=1}^L \sqrt{\frac{1}{K} \sum_{k=1}^K (X(l, k) - \hat{X}(l, k))^2}$$

Dove X e $\hat{X}$ rappresentano le grandezze di magnitudine log-spettrali di y e x , queste sono pari a:

$$X = \log_{10} |S(y)|^2$$

Dove $S(y)$ è la Short-Time Fourier Transform del segnale y con una *finestra di Hanning* di 2048 campioni, mentre l e k sono rispettivamente il numero di finestre ed il numero di frequenze [23].

Capitolo 4

Sperimentazione

L'obiettivo della sperimentazione è la valutazione delle prestazioni di una Generative Adversarial Network il cui Generatore è una *Rete Neurale Ricorrente* ed il cui Discriminatore è un *discriminatore multi-scala*.

4.1 Reti Neurali Ricorrenti

Le Reti Neurali Ricorrenti sono modelli che affondano le proprie radici negli studi di Rumelheart, Misnky e Papert [2]. Queste reti lavorano su sequenze di input uno alla volta: ad ogni step il modello riceve un nuovo input e una rappresentazione "nascosta" calcolata a partire dagli input precedenti (la cosiddetta "connessione ricorrente") [2]. L'output finale contiene informazioni sull'intera sequenza di input e le loro applicazioni spaziano dal Natural Language Processing fino a modelli autoregressivi per immagini [2]. Le reti ricorrenti non sono però prive di problematiche, la più importante è la possibilità che il modello "dimentichi" gli input che costituiscono la parte iniziale della sequenza [2]. Per far fronte a questa dinamica sono state introdotte delle versioni più *s sofisticate* dell'architettura: le *long short-term memory networks* (o LSTMs) e le *gated recurrent units* (o GRUs) [2].

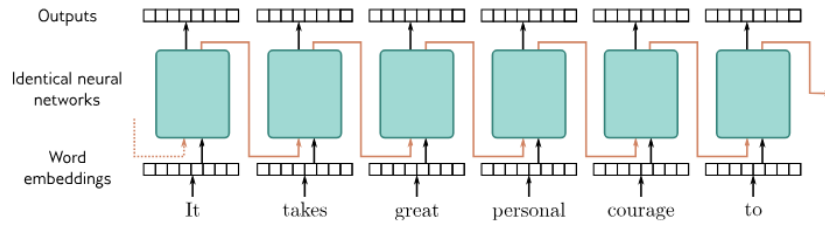


Figura 4.1: Utilizzo di una RNN nel Natural Language Processing. Gli embedding vengono dati in input ad una serie di reti identiche, ognuna delle quali produce due output: un embedding "di output" ed un embedding che viene dato in input alla rete successiva insieme alla prossima parola da processare. L'output finale dovrebbe dunque contenere informazioni sull'intera frase ma, come appena esposto, spesso le RNN tendono a dimenticare gradualmente i primi elementi della sequenza [2]

4.2 Architetture con molteplici Discriminatori

L'architettura originale di GAN proposta da Goodfellow et al. [8] prevede la presenza di un solo Generatore e di un singolo Discriminatore per poter modellare la logica avversariale come un gioco di minmax a due giocatori. Durugkar et al. [29] nel 2017 propongono una variazione dell'architettura originale introducendo la possibilità di avere N discriminatori distinti.

4.2.1 Architettura GMAN

L'architettura presentata da Durugkar et al. [29] introduce una novità nelle architetture per GANs nella forma dell'utilizzo di più discriminatori *distinti*. Per accogliere questa modifica è richiesta la riformulazione della funzione obiettivo del Generatore che diventa ora:

$$\min_G \max F(V(D_1, G), \dots, V(D_N, G))$$

dove ogni Discriminatore D_i dovrà comunque massimizzare la propria funzione $V(D_i, G)$ (non consentendo dunque alcun tipo di cooperazione).

F è la funzione che aggrega i feedback provenienti dai Discriminatori che saranno dati in input a G [29]:

- Con $F = \max$ il Generatore si allena sulla base dei feedback provenienti dal miglior discriminatore;
- Con $F = \text{mean}$ il Generatore si allena sulla base dei feedback provenienti dall'ensemble dei discriminatori;

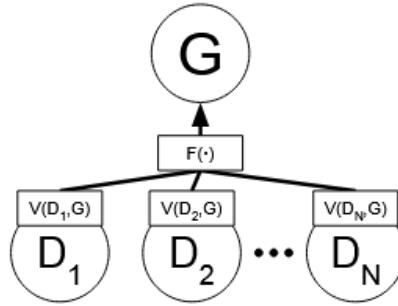


Figura 4.2: Rappresentazione della logica dietro l'architettura GMAN, i risultati dei discriminatori diventano input per la funzione F che stabilisce la logica di addestramento del generatore.

Nella nuova configurazione "multi-discriminatore" massimizzare $F_G(V_i)$ con $F = \max$ equivale a ottimizzare V con diverse ripartenze in parallelo e presentare

$$\max_{i \in 1, \dots, N} V(D_i, G)$$

come loss risultante al Generatore [29].

In questo modo G è forzato a produrre campioni ad alta fedeltà per soddisfare tutti gli N discriminatori, nella pratica la convergenza non è quasi mai raggiunta [29]. Nel lavoro è inoltre espressa la necessità di regolare quanto D sia "critico" nei confronti dei campioni prodotti da G , per questo motivo sono introdotte delle versioni "ammorbidite" delle medie pitagoriche parametrizzate da λ , dove $\lambda = 0$ equivale a "mean" e $\lambda \rightarrow \infty$ equivale a "max" [29].

Le formule sono le seguenti:

$$AM_{soft}(V, \lambda) = \sum_i^N w_i V_i$$

$$GM_{soft}(V, \lambda) = -\exp\left(\sum_i^N w_i \log(-V_i)\right)$$

$$HM_{soft}(V, \lambda) = \left(\sum_i^N w_i V_i^{-1}\right)^{-1}$$

dove

$$w_i = \frac{e^{\lambda V_i}}{\sum_j e^{\lambda V_j}}$$

L'approccio presentato è quello di porre inizialmente $\lambda \approx 0$ (simulando dunque una media) così da ottenere feedback costruttivi per il Generatore, l'ensemble di discriminatori riuscirà inoltre a diminuire la varianza dei feedback e aiutando gli stessi a definire un decision boundary ottimale; al migliorare delle prestazioni dei discriminatori segue un aumento del valore di λ così da affinare l'addestramento di G [29]. Sperimentalmente si è dimostrato come questa architettura arrivi più rapidamente a convergenza e riduca la varianza della funzione obiettivo nel tempo.

4.2.2 Discriminatori multi-scala per immagini

L'architettura presentata da Wang et al. [30] mira a produrre immagini fotorealistiche ad alta risoluzione a partire da mappe semantiche utilizzando le Generative Adversarial Networks. Una delle idee chiave del modello risiede nella scelta di implementare una logica "multi-discriminatore" in maniera innovativa: per differenziare immagini ad alta risoluzione da quelle sintetiche il discriminatore deve possedere un grande campo recettivo e per far fronte a questa necessità bisogna aumentare la profondità della rete oppure aumentare la dimensione del kernel di convoluzione, portando a possibili fenomeni di overfitting e ad un incremento nella quantità di memoria necessaria durante l'addestramento [30]. Per risolvere la problematica esposta viene presentata una soluzione basata su tre discriminatori identici che operano su scale diverse, ognuno con un determinato fattore di sottocampionamento dell'immagine.

Il Discriminatore che opera sulla scala più grossolana ha il campo recettivo più grande e questo consente di avere una visione d'insieme migliore, guidando il generatore verso la sintesi di output globalmente più consistenti; il Discriminatore che opera sulla scala più fine può invece cogliere i dettagli che altrimenti sarebbero "invisibili" consentendo al Generatore di ridurre la possibilità di creare artefatti visivi [30]. Nell'articolo si fa inoltre

riferimento a possibili pattern o artefatti che possono insorgere se non si fa uso dell'approccio multi-scala, una caratteristica già espressa in precedenza durante la trattazione sulle GANs.

L'utilizzo di più discriminatori porta necessariamente a dover riformulare il problema, questa volta come:

$$\min_G \max_{D_1, D_2, D_3} \sum_{k=1,2,3} \mathcal{L}_{GAN}(G, D_k)$$

Un'altra novità presentata nel lavoro è costituita dall'introduzione di una *feature matching loss* per stabilizzare l'addestramento [30]. Per ciascun discriminatore D_k vengono estratte le mappe di feature secondo la formula:

$$\mathcal{L}(G, D_k) = \mathbb{E}_{(s,x)} \sum_{i=1}^T \frac{1}{N_i} [\|D_k^{(i)}(s, x) - D_k^{(i)}(s, G(s))\|]$$

Dove T è il numero di layer totali, N_i è il numero di elementi ad ogni strato, $D_k^{(i)}$ è l'operazione di estrazione di feature al layer i -esimo del k -esimo discriminatore.

La funzione obiettivo della rete combina dunque la loss della GAN e la feature matching secondo la formula [30]:

$$\min_G ((\max_{D_1, D_2, D_3} \sum_{k=1,2,3} \mathcal{L}_{GAN}(G, D_k)) + \lambda \sum_{k=1,2,3} \mathcal{L}_{FM}(G, D_k))$$

Il parametro λ controlla l'importanza dei due addendi.

Sperimentalmente si è dimostrato come l'approccio multi-scala produca immagini più gradevoli agli occhi di soggetti umani, inoltre l'utilizzo di una loss combinata (GAN + feature matching) permette di ottenere miglioramenti sostanziali nella generazione [30].

4.2.3 Discriminatori multi-scala per audio

La rete MelGAN introdotta da Kumar et al. in [31] è una innovativa rete neurale per la generazione di forme d'onda che sfrutta i mel-spectrogram e un discriminatore multi-scala per raggiungere risultati di elevata qualità in vari task di generazione [31]. Il Generatore consiste in una rete convoluzionale feed forward che prende in input un mel-spectrogram e restituisce una forma d'onda in output; poichè lo spettrogramma ha una risoluzione temporale inferiore rispetto all'output la rete effettua inizialmente operazioni di sovra-campionamento in più stadi. Ogni stadio di sovra-campionamento

consiste in una convoluzione (con kernel size pari a due volte il valore dello stride per evitare i "checkerboard patterns" [31]) seguita da uno stack residuale che aumenta il campo recettivo fino a 27 timesteps per catturare le correlazioni a lungo raggio senza aumentare le dimensioni della rete [31].

Il Discriminatore segue invece la logica multi-scala introdotta in [30] con tre Discriminatori che operano su diverse scale di risoluzione: D_1 lavora sull'audio grezzo e può per tanto cogliere dettagli "globali", D_2 e D_3 lavorano su audio con fattori di sotto-campionamento pari a 2x e 4x, l'operazione di sotto-campionamento avviene attraverso un average pooling [31]. I tre discriminatori lavorano inoltre secondo una logica "window based" utilizzando delle convoluzioni raggruppate e potendo quindi usare una elevata kernel size ed un basso numero di parametri [31]. Rispetto ad un discriminatore standard questo approccio consente di far concentrare l'addestramento su "blocchi" di audio più piccoli riuscendo inoltre a cogliere le strutture ad alta frequenza e riducendo l'insorgere di fenomeni inattesi come gli artefatti [31]. In un approccio con un solo discriminatore questo rischierebbe di non cogliere possibili artefatti in quanto si potrebbe concentrare solo su pattern globali o solo su dettagli più fini, inoltre un discriminatore con un campo recettivo elevato potrebbe portare a fenomeni di overfitting come già evidenziato da Wang et al. [30]. Ogni Discriminatore effettua una prima convoluzione per l'estrazione di feature, l'input è sottocampionato altre quattro volte e per ogni iterazione è estratta una nuova feature map, segue un'ulteriore convoluzione (e relativa estrazione della feature map) e infine con l'ultimo layer viene restituito l'output. [31]

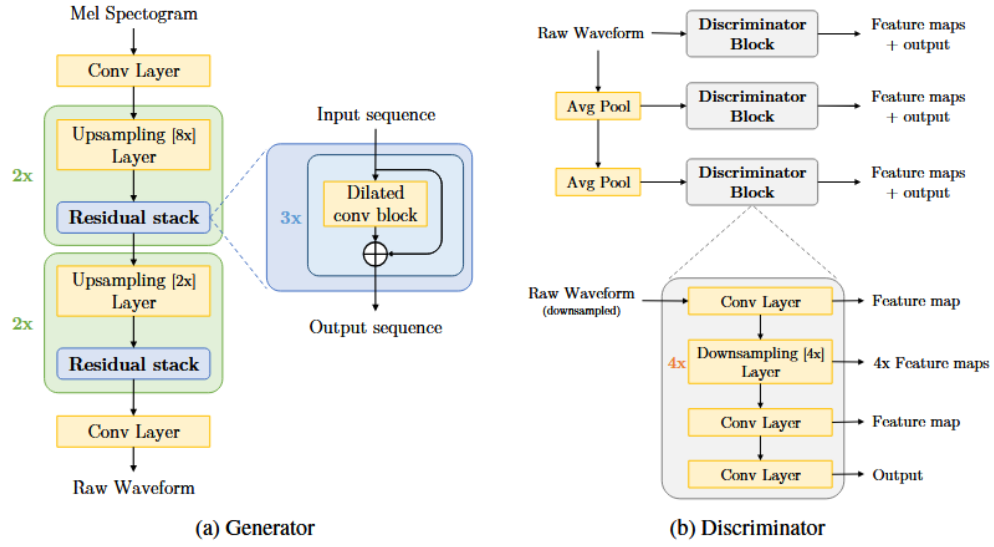


Figura 4.3: Struttura della rete MelGAN [31]

La funzioni obiettivo per il generatore e per i discriminatori sono le seguenti [31]:

$$\min_G \mathbb{E}_{s,z} \left[\sum_{k=1,2,3} -D_k(G(s, z)) \right]$$

$$\min_{D_k} \mathbb{E}_x [\min(0, 1 - D_k(x))] + \mathbb{E}_{s,z} [\min(0, 1 + D_k(G(s, z)))], \forall k = 1, 2, 3$$

Dove x è la forma d'onda, s è il mel-spectrogram e z è un vettore di rumore gaussiano. In aggiunta, come già sperimentato da Wang et al. [30], viene presa in considerazione la feature matching loss con formula:

$$\mathcal{L}_{FM}(G, D_k) = \mathbb{E}_{x,s \sim p_{data}} \left[\sum_{i=1}^T \frac{1}{N_i} \|D_k^{(i)}(x) - D_k^{(i)}(G(s))\| \right]$$

L'obiettivo finale del Generatore diventa dunque [31]:

$$\min_G \left(\mathbb{E}_{s,z} \left[\sum_{k=1,2,3} -D_k(G(s, z)) \right] + \lambda \sum_{k=1}^3 \mathcal{L}_{FM}(G, D_k) \right)$$

Con $\lambda = 10$.

La rete dimostra sperimentalmente si avere un buon compromesso qualità/velocità, inoltre la scelta dell'approccio multi-scala si rivela cruciale nell'aumento della qualità percepita degli output e nella riduzione della presenza di possibili artefatti [31].

4.3 Specifiche di sperimentazione

4.3.1 Il Free Music Archive dataset

Il dataset usato nell'addestramento è il "*Free Music Archive*" (o "FMA"), progettato per task di valutazione da parte di algoritmi musicali o per training di modelli di machine learning [32]. Sono contenute un totale di 106.574 tracce provenienti da 14.854 album di 16.341 artisti di 161 generi musicali diversi; tutti i contenuti sono in formato .mp3 con un tasso di campionamento pari a 44.1 kHz ed un bitrate medio di 263 kbit/s. L'intero dataset è un dump del sito "Free Music Archive" che raccoglie brani rilasciati con licenze permissive come Creative Commons. Esistono più sottogruppi del FMA, ognuno con un peso ed un numero di tracce diverso, nella sperimentazione è stato usato il "FMA Small" contenente 8000 clip musicali (1000 per ognuno degli 8 generi più popolari) da 30 secondi ognuna; le versioni più grandi offrono un numero maggiore di clip con un conseguente aumento dello spazio di archiviazione utilizzato [32].

4.3.2 Implementazione del Discriminatore multi-scala per Audio Super Resolution

Il Discriminatore della rete è composto da due classi: un `MultiScaleDiscriminator` che crea e gestisce tre `SubDiscriminator` su scale di risoluzione diverse, i quali implementano a loro volta la logica di discriminazione.

4.3.3 SubDiscriminator

Ogni `SubDiscriminator` prende in ingresso un tensore STFT complesso e implementa la seguente pipeline: viene creato un vettore "channels" per definire quante feature map vengono create ad ogni layer di convoluzione; il numero base di canali è 32 e per ogni layer questo numero è moltiplicato per un fattore progressivo [1,2,4,8,8] e per un fattore "channels_mult" che varia sulla base della scala su cui opera il Discriminatore. All'aumentare dei canali aumenta anche il numero di feature estratte. Opzionalmente si può decidere di applicare la normalizzazione spettrale ai layer di convoluzione.

L'input deve essere un tensore complesso con attributi reali (.real) e immaginari (.imag)

organizzati come [batch, freq_bins, time_frames], le due componenti sono successivamente trasposte in [batch, time_frames, freq_bins] a cui viene applicata successivamente un'operazione di "unsqueeze" per ottenere la configurazione [batch, 1, time_frames, freq_bins]. Seguono due stream paralleli per l'encoding della parte reale e della parte immaginaria: ognuno composto da 4 layer di convoluzione bidimensionale, dopo ogni strato viene applicata una BatchNorm2d e una funzione di attivazione LeakyReLU. Dopo il quarto layer si applica un dropout. Le feature estratte ai layer più bassi dell'encoder (r4 e i4) vengono successivamente concatenate, questo risultato subisce una operazione di fusione tramite convoluzione. L'oggetto passa per un'ultima convoluzione che restituisce la mappa di punteggi finale. L'output del discriminatore è dunque composto da 3 elementi: una mappa di punteggi che valuta singole regioni dello spettro, le feature estratte dagli encoder e la fusione di r4 e i4.

4.3.4 MultiScaleDiscriminator

Il MultiScaleDiscriminator crea un numero di discriminatori pari a "num_scales", negli esperimenti ne vengono creati tre, per ogni scala "i" viene creato un SubDiscriminator con un moltiplicatore del numero di canali pari a 2^i . Tra l'elaborazione di una scala di risoluzione l'altra viene applicata al tensore STFT complesso una combinazione di trasposizione e di unsqueeze. Le due componenti (reale ed immaginaria) vengono successivamente sotto-campionate per poi ricostruire il tensore complesso da dare in input al prossimo discriminatore. MultiScaleDiscriminator restituisce in output la lista di mappe di punteggio dei tre discriminatori e le loro feature map intermedie.

4.3.5 Specifiche dell'addestramento

La rete GAN è composta dalla RNN e dal MultiScaleDiscriminator ed è stata allenata per 105 epoche, una volta inizializzate le due reti contestualmente sono creati due ottimizzatori "Adam".

Le funzioni di loss utilizzate sono la Mean Squared Error e la Spectral Coherence Loss per il Generatore, dove quest'ultima calcola le differenze temporali tra la predizione e la

ground truth e vi applica la MSE, penalizzando ricostruzioni inesatte delle componenti ad alta frequenza. La funzione di loss del Discriminatore è composta da tre componenti: una funzione che può essere scelta tra LSGAN, WGAN e BCE, rispettivamente con formula:

$$\min_D V_{LSGAN}(D) = \frac{1}{2} \mathbb{E}_{x \sim p_{data}(x)} [(D(x) - b)^2] + \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [(D(G(z)) - a)^2]$$

$$\mathcal{L}_D = \mathbb{E}_{x \sim \mathbb{P}_r} [f_w(x)] - \mathbb{E}_{z \sim p_z} [f_w(G(z))].$$

$$L = -\frac{1}{N} \sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

In aggiunta alla scelta della funzione di loss vi è l'implementazione di una feature matching loss e di una eventuale penalità del gradiente (che può essere omessa).

La loss totale del Generatore è dunque pari a:

$$\mathcal{L}_G = \alpha \mathcal{L}_{MSE} + \beta \mathcal{L}_{coh} + \gamma \mathcal{L}_{adv}$$

Dove $\mathcal{L}_{adv}$ è la loss "avversariale" generata dal discriminatore.

α, β, γ sono invece i pesi assegnati ad ognuno dei componenti della loss.

La loss totale del Discriminatore è invece pari a:

$$\mathcal{L}_D = \sum_{n_scale} \mathcal{L}_{adv} + \lambda_{FM} \cdot \mathcal{L}_{FM} + \lambda_{GP} \cdot \mathcal{L}_{GP}$$

λ_{FM} e λ_{GP} sono i pesi assegnati ad ognuno dei componenti della loss.

Il ciclo di train consiste nell'aggiornamento del Discriminatore seguito da un aggiornamento del Generatore, viene effettuata una validazione su un piccolo batch ogni 5 epoche e periodicamente (ogni 10 epoche) viene creato un file di checkpoint contenente lo stato di G, di D, degli ottimizzatori e la storia di addestramento attuale per l'eventuale ripristino dello stato della rete.

Nel codice sono presenti ottimizzazioni quali la pulizia periodica della cache della GPU e il limite alla RAM utilizzabile per evitare possibili crash del device durante l'esecuzione.

L'addestramento della rete è stato eseguito su piattaforma "Kaggle" utilizzando come device 2 GPU Nvidia Tesla T4 e una CPU a 4 core con 30GB di RAM.

Di seguito sono riportati i valori degli iperparametri della rete:

- Tasso di apprendimento del Generatore: 2^{-4} ;

- Tasso di apprendimento del Discriminatore: 2^{-4} ;
- Batch size: 2;
- Accumulation steps: 4;
- α : 0.6;
- β : 1.0;
- γ : 0.6;
- λ_{FM} : 10.0;
- λ_{GP} : 3.0;

4.4 Risultati

L'addestramento della rete è stato valutato sulla base delle due metriche presentate in precedenza: la Log-Spectral-Distance e il Signal-to-Noise Ratio. Di seguito sono riportati i grafici di andamento dei valori delle funzioni di loss realizzati grazie ai file di checkpoint:

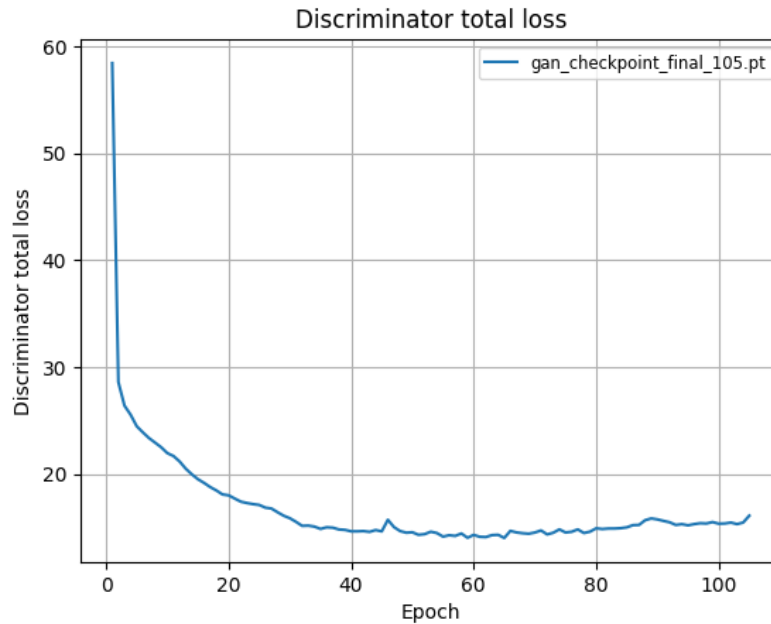


Figura 4.4: Andamento della loss del Discriminatore

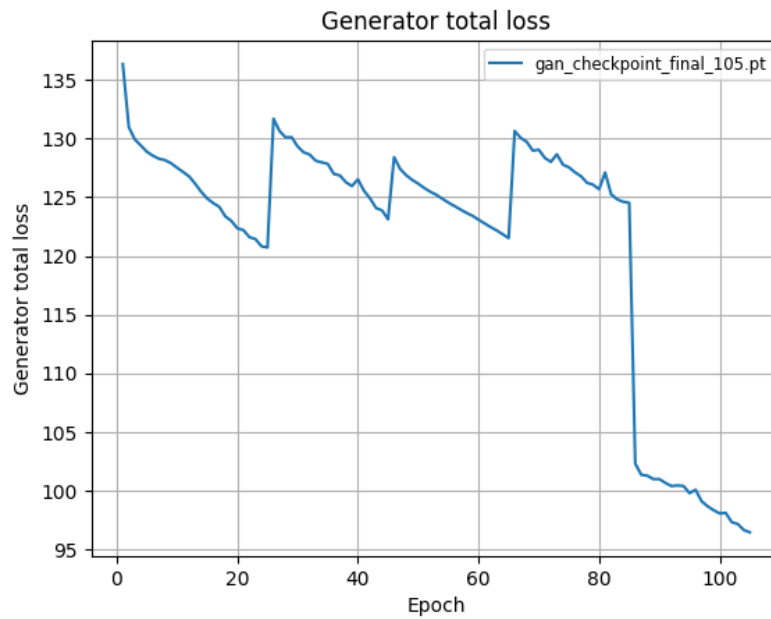


Figura 4.5: Andamento della loss del Generatore, i repentini incrementi della loss sono da imputare al caricamento dei pesi a partire dai file di checkpoint

Per quanto concerne le metriche, i valori sono i seguenti:

- Signal-to-Noise Ratio: 0.33729523;
- Signal-to-Noise Ratio (dB): ≈ -4.71989800522 ;
- Log-spectral Distance: 0.61798366;

Il valore di SNR in decibel fornisce una visione più chiara dei risultati ottenuti: la potenza del segnale ricostruito è inferiore rispetto alla componente identificata come rumore residuo, il modello ha dunque un ampio margine di miglioramento. I risultati della sperimentazione rappresentano un primo passo promettente, la rete può sicuramente migliorare con una gestione più fine dei valori degli iperparametri quali learning rate e il peso dei singoli componenti delle funzioni di loss oppure dall'utilizzo di un dataset più ampio, in questo caso è stato utilizzato "FMA Small" per via dei limiti imposti dalla piattaforma Kaggle sull'utilizzo dello spazio di archiviazione.

Capitolo 5

Conclusioni

Il lavoro ha affrontato il problema dell'Audio Super Resolution proponendo una innovativa soluzione basata su una GAN composta da una Rete Ricorrente e da un Discriminatore Multi-Scala.

Sono stati trattati i principali ambiti coinvolti dal problema: Digital Signal Processing e Deep Learning, attraverso una analisi degli elementi fondamentali di entrambi i campi di studio e degli aspetti più specifici strettamente legati all'ASR.

Nel primo capitolo è stata fornita una base teorica sulle reti neurali ed il deep learning, con una attenzione particolare rivolta verso le GANs, attraverso una trattazione che ne mostra sia i punti di forza che le possibili debolezze.

Nel secondo capitolo sono stati trattati gli aspetti fondamentali dell'elaborazione dei segnali digitali, necessari per la comprensione del problema e per la corretta gestione degli aspetti implementativi del codice.

Nel capitolo successivo sono state esposte le principali architetture di rete proposte per il problema dell'ASR presenti in letteratura e le metriche di valutazione delle performance della rete generatrice.

Nel quarto capitolo sono state presentate le architetture di rete, non tutte legate all'Audio Super Resolution, che fanno uso della logica multi-discriminatore e multi-scala, è stata successivamente descritta l'implementazione della rete con i relativi iperparametri e le specifiche dell'addestramento, in ultima battuta sono stati trattati i risultati della rete.

Il lavoro ha dimostrato come la combinazione di RNN e Discriminatore multi-scala possa essere una scelta corretta per via delle basi su cui poggia la propria

implementazione, principalmente legate all'approccio multi-scala come strumento per la riduzione della quantità di artefatti presenti nella ricostruzione, come già mostrato sperimentalmente in letteratura.

Pur non avendo raggiunto le performance ottimali, i risultati costituiscono una prima base sperimentale che può essere sviluppata in futuro.

Bibliografia

- [1] McCulloch, W.S., Pitts, W. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics* 5, 115–133 (1943). <https://doi.org/10.1007/BF02478259>
- [2] Prince, J.D. Simon. *Understanding Deep Learning*. MIT Press (2023).
- [3] Cybenko, G. Approximation by superpositions of a sigmoidal function. *Math. Control Signal Systems* 2, 303–314 (1989). <https://doi.org/10.1007/BF02551274>
- [4] Householder, A.S. A theory of steady-state activity in nerve-fiber networks: I. Definitions and preliminary lemmas. *Bulletin of Mathematical Biophysics* 3, 63–69 (1941). <https://doi.org/10.1007/BF02478220>
- [5] K. Fukushima, Visual Feature Extraction by a Multilayered Network of Analog Threshold Elements, in *IEEE Transactions on Systems Science and Cybernetics*, vol. 5, no. 4, pp. 322-333, Oct. 1969, doi: 10.1109/TSSC.1969.300225
- [6] Fukushima, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybernetics* 36, 193–202 (1980). <https://doi.org/10.1007/BF00344251>
- [7] Rumelhart, D., Hinton, G. & Williams, R. Learning representations by back-propagating errors. *Nature* 323, 533–536 (1986). <https://doi.org/10.1038/323533a0>
- [8] Goodfellow, Ian; Pouget-Abadie, Jean; Mirza, Mehdi; Xu, Bing; Warde-Farley, David; Ozair, Sherjil; Courville, Aaron; Bengio, Yoshua (2014). Generative Adversarial Nets. *Proceedings of the International Conference on Neural Information Processing Systems (NIPS 2014)*. pp. 2672–2680.

- [9] Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. (2016). Improved techniques for training gans. *Advances in neural information processing systems*, 29.
- [10] Arjovsky, M., Chintala, S. & Bottou, L.. (2017). Wasserstein Generative Adversarial Networks. *Proceedings of the 34th International Conference on Machine Learning*, in *Proceedings of Machine Learning Research* 70:214-223.
- [11] Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30.
- [12] Steven W. Smith. 1997. *The scientist and engineer's guide to digital signal processing*. California Technical Publishing, USA.
- [13] C. E. Shannon, "Communication in the Presence of Noise," in *Proceedings of the IRE*, vol. 37, no. 1, pp. 10-21, Jan. 1949, doi: 10.1109/JRPROC.1949.232969.
- [14] Alan V. Oppenheim, Ronald W. Schaffer, and John R. Buck. 1999. *Discrete-time signal processing* (2nd ed.). Prentice-Hall, Inc., USA.
- [15] Leon Cohen. 1995. *Time-frequency analysis: theory and applications*. Prentice-Hall, Inc., USA.
- [16] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," in *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51-83, Jan. 1978, doi: 10.1109/PROC.1978.10837.
- [17] Douglas O'Shaughnessy, *Speech Communication: Human and Machine*. Reading, Massachusetts: Addison-Wesley Publishing Company, 1987
- [18] Volkmann, J. & Stevens, S. & Newman, E.. (2005). A Scale for the Measurement of the Psychological Magnitude Pitch. *The Journal of the Acoustical Society of America*. 8. 208-208. 10.1121/1.1901999.

- [19] J. B. Allen and L. R. Rabiner, "A unified approach to short-time Fourier analysis and synthesis," in *Proceedings of the IEEE*, vol. 65, no. 11, pp. 1558-1564, Nov. 1977, doi: 10.1109/PROC.1977.10770.
- [20] J. Makhoul and M. Berouti, "High-frequency regeneration in speech coding systems," *ICASSP '79. IEEE International Conference on Acoustics, Speech, and Signal Processing*, Washington, DC, USA, 1979, pp. 428-431, doi: 10.1109/ICASSP.1979.1170672.
- [21] Kuleshov, Volodymyr & Enam, S. & Ermon, Stefano. (2017). "Audio Super Resolution using Neural Networks". 10.48550/arXiv.1708.00853.
- [22] Simona Nisticò, Luigi Palopoli, and Adele Pia Romano. 2022. Audio Super-Resolution via Vision Transformer. In *Foundations of Intelligent Systems: 26th International Symposium, ISMIS 2022, Cosenza, Italy, October 3–5, 2022, Proceedings*.
- [23] Zhang, Kexun & Ren, Yi & Xu, Changliang & Zhao, Zhou. (2021). WSRGlow: A Glow-Based Waveform Generative Model for Audio Super-Resolution. 1649-1653. 10.21437/Interspeech.2021-892.
- [24] Mandel, Moshe & Tal, Or & Adi, Yossi. (2022). AERO: Audio Super Resolution in the Spectral Domain. 10.48550/arXiv.2211.12232.
- [25] Lee, Junhyeok & Han, Seungu. (2021). NU-Wave: A Diffusion Probabilistic Model for Neural Audio Upsampling. 1634-1638. 10.21437/Interspeech.2021-36.
- [26] Don H. Johnson (2006) Signal-to-noise ratio. *Scholarpedia*, 1(12):2088.
- [27] Gray, Augustine H. and J. Markel. "Distance measures for speech processing." *IEEE Transactions on Acoustics, Speech, and Signal Processing* 24 (1976): 380-391.
- [28] Hochreiter, Sepp & Schmidhuber, Jürgen. (1997). Long Short-Term Memory. *Neural Computation*. 9. 1735-1780. 10.1162/neco.1997.9.8.1735.

- [29] Durugkar, Ishan & Gemp, Ian & Mahadevan, Sridhar. (2016). Generative Multi-Adversarial Networks. 10.48550/arXiv.1611.01673.
- [30] T. -C. Wang, M. -Y. Liu, J. -Y. Zhu, A. Tao, J. Kautz and B. Catanzaro, "High-Resolution Image Synthesis and Semantic Manipulation with Conditional GANs," 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 2018, pp. 8798-8807, doi: 10.1109/CVPR.2018.00917.
- [31] Kumar, K., Kumar, R., De Boissiere, T., Gestin, L., Teoh, W. Z., Sotelo, J., ... & Courville, A. C. (2019). Melgan: Generative adversarial networks for conditional waveform synthesis. *Advances in neural information processing systems*, 32.
- [32] Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2016). FMA: A dataset for music analysis. *arXiv preprint arXiv:1612.01840*.